Order For A PDF

Analysis

longList

Idea	Explanation	Rank
BarFight	Street Fighter esque 3rd person 2d fighter. Player vs bot with several attacks	1
Smart Speaker	Laptop based smart assistant using audio to text processing libraries. The main code (for marks) would stem from the functions and processing of the returned text. This could be system operations such as opening applications, or even navigation (through Hyprland WM)	2
FTP Server	Home FTP server for file transfer and access remotely	3
Spotify Client	Web/Desktop/IOS App making use of the Spotify Developer API to get user data such as playlists. Either show stats or play music.	4
Potion Brewing	Mix different ingredients with various values for properties in a cauldron. The values of each ingredient are totalled in the potion changing its display and effects.	5
Bureau De Change	Uses Fixer API to get real time currency values. Interface allows viewing and purchase of multiple currencies and amounts.	6
Hole In The Fence	Farming wall blocker simulator. Sheep object starts in enclosure with a hole in the fence(s). Each turn the user places a wall to entrap the sheep, calculating the shortest path to escape, and prevent escape.	7
Weather App	Dysfunctional weather app which only displays weather where the user isn't	8
The Weakest Link	Social networking for the unemployed	9
General Quiz	Score and timer tracked, questions given by the Trivia API.	10

shortList

Idea	Rank
Further Exploration of BarFight	1
Further Exploration of Smart Speaker	2
Further Explanation of Potion Brewing	3
Further Explanation of Spotify Client	4
Further Explanation of FTP Server	5

Further Explanation of FTP Server

Must	Would	Could
Store Files Remotely		Use services like Tailscale to interface with multiple devices
Be Accessible Remotely	Sync file data with other connected devices	
	Show metrics across online devices	Graph the different contributions of each device similar to git commits

Potential Challenges	Solutions
It may be difficult to extend the features of the server	Extend the features of the app
Hardware could be expensive to source	
There may be a lack of space for self written code	Focus the self written code on a GUI web app

Further Exploration of BarFight
In the assumption that this is done in Greenfoot, certain features may prove harder to implement. Thus this document seeks to preemptively address difficulties.

Features List

Must Include	Should Include	Could Include
Bots to play against	Different attacks	Interactive Environment/Props
Bar like setting	Multiple enemies	Alcohol
Stamina bar	Consecutive hit counter	Powerups
Hit delay		Impact frames
Small Stun upon receiving hit		
Health		

Bots

The easiest thing to do for this, or the bare minimum, would be to have them swing randomly.
This could be extended by making them **react on certain conditions**, such as the relative distance of the player. This could be expanded on by having these methods and attributes in a `bot` class. It would have limiting attributes such as **reaction time**. This would run a for loop to represent hesitation.
The most extreme fulfilment of this idea would be to use libraries to train the ai. This may not be possible due to the limitations of Greenfoot.

Stamina Bar

As something for both players, this would be an attribute stored in a fighter class, housing **both the player and the bot**.
The most reasonable method of implementation for this would be to impose an **ability cost** to each. For example performing an uppercut may cost an arbitrary 10 `stamina`.

Stun

This would be similar to the **hesitation loop** from the bot.
The easy implementation would be identical to stamina control, with each attack adding to a `player_stun` attribute.
In extending this, multiple attacks in combination could continue to increase this stun number to a threshold. This would hinge on the condition of `time_since_hit > X`. Then the loop would commence, stunning the player.

Punch

This would be a significant challenge due to animating in Greenfoot. Not only this, but it should have collision. My idea for this is that it would come in two parts: one player animation, and one created projectile with collision. To avoid the activating player from also colliding with this, I would have a class for attacks, split by Player and Bot. The movement for this "attack" object largely depends on the intended complexity. It could simply move 5 units in a straight line an then disappear. The alternative is to have it loosely follow gravity, posing its velocity as a vector such as `10i-2j`.

Analysis

My current idea for this project is for it to provide a simple, retro-themed fighting game for one player. It will involve using different moves, each controlled by appropriate key bindings, to tackle various opponents with equal power. Opponents would be generated based on the player's options with extra attributes such as reaction time to make it fair. Further scaling of the project would include a difficulty slider or even different rounds fighting enemies of greater and greater strength or speed. This would challenge the user, making the game fun and engaging to play.

My current knowledge of the games industry shows that similar games are underrepresented, despite their undisputed popularity amongst all audiences. Current examples include Street Fighter and Mortal Kombat. Other games with desirable features for this product are Gladio Mori, Half Sword and Watermelon Sandbox. Whilst two of these are 3d, I admire their menus, theming and core mechanics. This project has the goal of filling this niche in a scalable manor. This design could remove the problem of boredom from the user as they would have never truly beaten it.

The embarkment of this project excites me as I can appreciate the dedication and creativity in the games industry. By completing this relatively small project, I would be able to use it for developing my skills. An ideal development engine should be incredibly minimal, thus allowing me to use all of my skills organically, rather than having them built in. Additionally, it ought to be compatible across PC operating systems such as MacOS, Debian and NixOS. This would garner a wider player base and expand the clientele of the game.

For this project to be successful, it should allow a player to

Designing

developmentPlan

Development Plan

Stage	Involves	Success Criteria	Success Indicator	desirable essential
1	Creating the player.	They should be able to move left, right and up with corresponding input keys. They should also be able to use attacks which briefly change the sprite. The data such as power and knockback of these attacks would not be of concern at this stage.	- [ ] Player Can move - [ ] Player can attack - [ ] Player jumps and returns to ground	essential
2	Creating Enemy	The Enemy actor is added to the scenario with basic movement and abilities to match the player. Its actions such as `move` should be controlled by monitored events such as Player proximity.	- [ ] Enemy can move - [ ] Enemy can attack - [ ] Enemy responds to Player events	essential
3	Setting up environment data collection.	Greenfoot world contains getters for data such as player health, stamina and distance from enemy. This is so that the enemy class can use these as events to act upon.	- [ ] Getters return accurate information about Player	essential
4	Creating HUD	The game should now have a HUD to display the player and enemy's health and stamina . The bar should deplete the lower the value represented. The health bars should hover above each characters' head. The stamina and cooldown bar (unique to player) should sit in the top right hand corner. A pause/play button be added also in the top left to start or stop the game.	- [ ] Health Bar follows player/enemy - [ ] Stamina bar in corner - [ ] Cooldown bar shown when necessary (in use) - [ ] Bars show accurate information - [ ] Game can be stopped/started	essential with desirable aspects such as floating healthbar
5	Start Menu	The game should load into the `Start Menu` , with options including start the game, change difficulty and quit. Difficulty slider should modify variables in Enemy class such as strength, speed, stamina. Start starts game.	- [ ] Start button starts game - [ ] Difficulty slider increases enemy power	essential
6	Better Attacks	Attack functions are populated with data for damage and knockback. For the player this is a base value, for the enemy this is random as above.	- [ ] Attack from either lowers health - [ ] Attack from	desirable

Stage	Involves	Success Criteria	Success Indicator	desirable essential
		Damage and knockback are to be taken using setters such that Player can apply knockback to Enemy.	either has damage proportional knockback	
7	Condition Checking	Basic condition checking such as `if(player.health < 0) {player.die;}`. Extends to stamina (limit attack), cooldown.	- [ ] Characters die - [ ] Attacks remove stamina such that they cannot be used constantly - [ ] Attacks have effective cooldowns	essential
8	Enemy Expansion	The enemy loads in with random variables within an appropriate range. This includes size (Greenfoot.imageSize), strength (within bounds relative to difficulty) and speed.	- [ ] Enemy size varies - [ ] Enemy speed varies - [ ] Enemy strength varies	desirable

Justifications

1

This is the most crucial stage, due to it being the crux of the whole game. By doing this first, I will be able to test almost everything else.

2

This stage is necessary for properly wiring the enemy. For testing, this will allow me to test mechanisms such as attacking, character-character collision.

3

By doing this, I can properly wire up the enemy. Furthermore, the data from the getters I plan to implement will serve as useful tests to ensure that everything is going as I'd like. The conditions in the enemy's code coming from here would make the game officially playable.

4

The HUD is the most stylistic element, and while helpful, not crucial to play-ability. This is why I have left this until the characters are set up. As well as this, by leaving this after the 3rd (data collection), all of the methods for returning data to be shown will be available, making this a theoretically smooth experience.

5

Like the HUD, this is not the most important element, hence it has been left to this cycle. I have also positioned it after designing the HUD so that the memory of the necessary code and methods to create a basic GUI will still be fresh in my memory.

6

The better attacks stage plans to populate the attacks with damage and knockback. This enables the stage after it, whilst also testing the health-bar. It does not go directly after the addition of the health-bar, allowing cycle 5 to take that place. This is because of the obvious link between both GUI based stages.

7

This stage is the final one until the game is entirely functional. It plans to introduce death and a limit on attacks enforced by stamina and cooldown. This stage is being implemented as one of the later stages due to it completing the structure of the game.

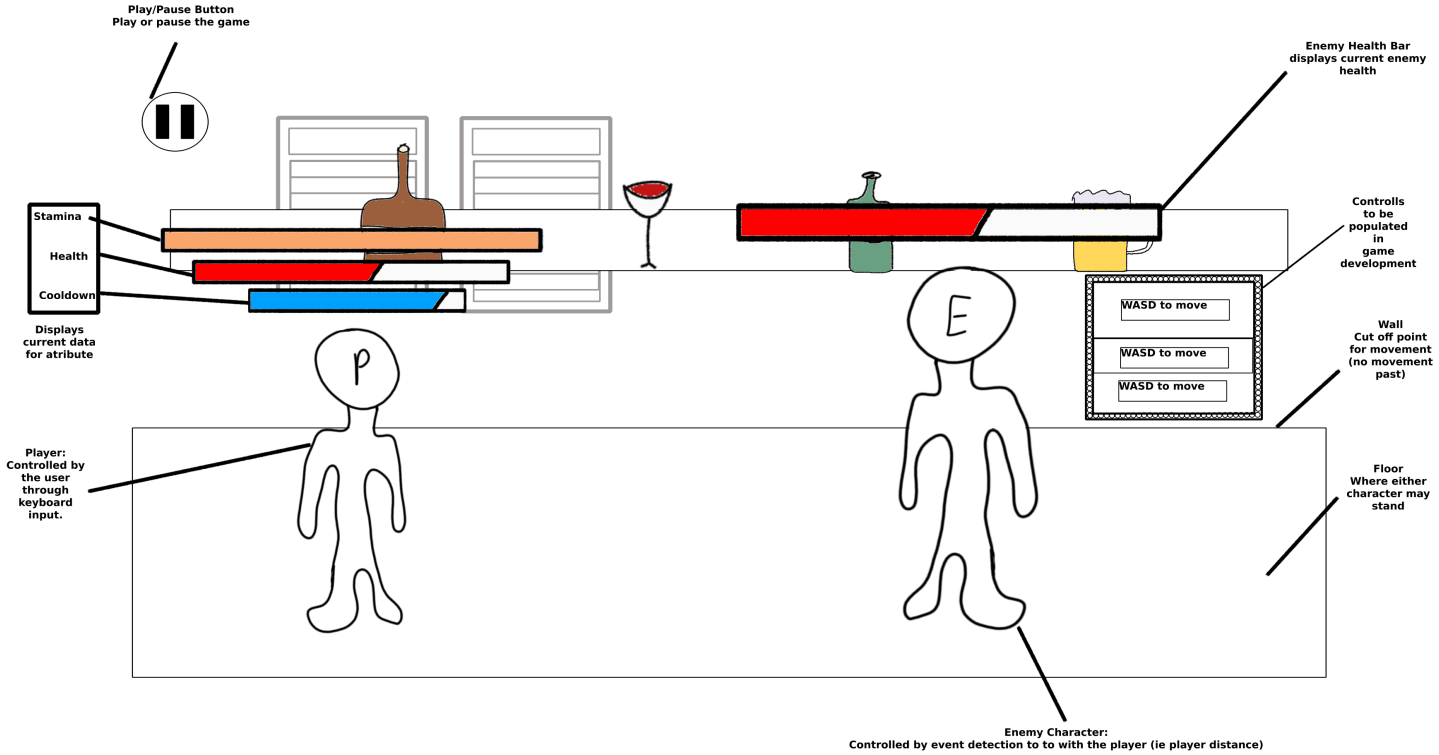
8

As the final stage, it represents a very small and slightly unnecessary aspect of the game, hence its position. This stage would be what I would like, and was lightly desired in my consumer research survey. Unfortunately, it cannot be justified to take precedence over the stages coming before it.

GUI

Main Screen

Bar Fight



The above is a rough outline of what I hope my project might become. As of this, details such as non-important colours including background and player sprites are not shown above.
This screen is the most detailed, because it will be displayed the majority of the time.

Bars

Above each will be coloured bars to display data about metrics corresponding to each character. The amount of the bar coloured in would be the percentage of the metric from full, for example if the player had 40 health remaining down from 100, the health bar would be 40% full.

Colours

The colours for each bar have been chosen to match the aesthetic of my game, whilst taking inspiration from other games of similar genres or categories. As such health would be red; stamina tan/peach; cooldown blue.

Visibility

As is above, more data would be available for the player's character than for the enemy. This adds more challenge to the game as well as authenticity with the player not knowing some data about their opponent such as stamina.

Furthermore, the cooldown bar would only be temporarily displayed. This being when it is needed (not 0). This would increase the player's view by only showing data useful at that moment.

Size Variance

The larger the value for a character's maximum or starting health, the larger the bar for that should be. In practice this would affect only the enemy once the relevant development cycle is complete.

Play/Pause Button

This button will sit in the top left corner, as my market research has led me to conclude is standard, due to the space freed up on the screen. Upon clicking, it will pause the game and change its sprite to reflect the state. In practice, pausing will look like stopping the movement of either player or enemy and halting the regeneration of stamina or cooldown.

Characters

The game will feature two characters, being the player and the enemy, both visible on the screen. Their movement will be shown live, making the game playable. I plan to draw sprites for each, matching them to the overall look of the game. The examples shown above are rough outlines of what I hope mine become. The size difference between the player and enemy character is intended, and hopefully will reflect plays with a higher difficulty, where the enemy is larger than the player.

Controls Screen

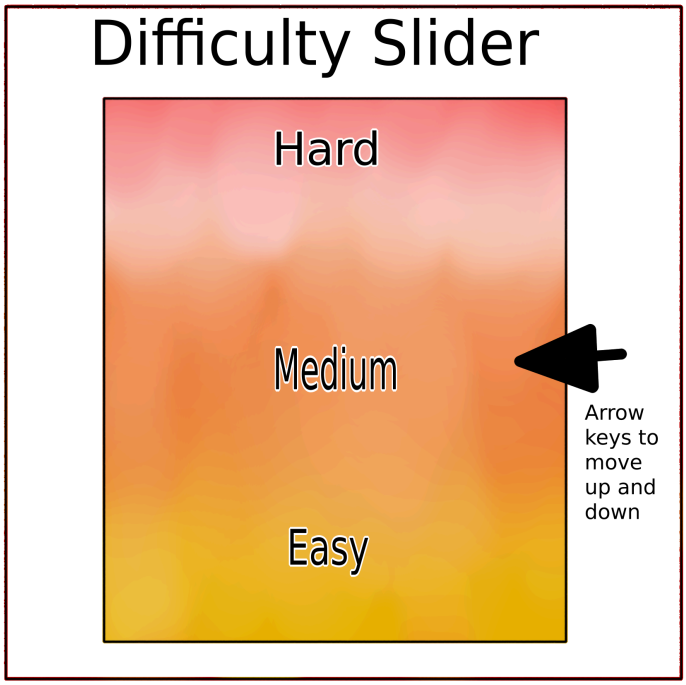
In the background of the scene will be a small display in the style of a bar's menu. It will have on it the various controls to perform basic actions such as move, punch, etc. I have chosen this style because it will reflect the general style of the game. This will increase the continuity of the game, making it feel better tuned and stylised.

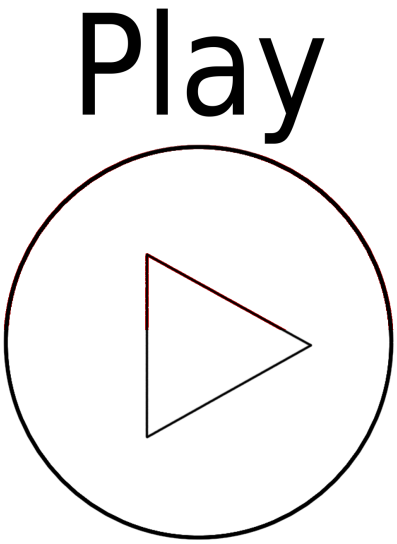
Background

The background in the image above is a rough simplification. I plan to keep elements like the bottles, shelves and drinks, but with a more full scene. The scene created by it should feel dark and quite moody, like a small pub.

Start Menu

Bar Fight





Like the main game GUI mock up, this is an abstraction of what I hope the actual thing looks like. Some features above are exaggerated so that they are quicker to find, and more obvious to make. Furthermore, there is no background image, hence the white background.
I have this main menu in order to bridge the gap between loading the game and fighting. This allows the player to spend time selecting a difficulty and preparing themselves.

Difficulty Slider

The difficulty slider above is coloured based on its difficulty. The cursor/selector would be set at medium by default, making the colour system intuitive to the user. The idea of moving the selector with arrow keys may be a little confusing, or not explicitly obvious, so I would like to have a label for this information. I also have deliberately put it on the left, such that the start screen is dominated by itself and the play buttons - the two main features of my start screen. As well as this, the position of the difficulty slider echoes games of a similar category, and the colour scheme would demonstrate this further. The result is a dynamic looking menu, with a clear colour scheme which evidences itself to its genre.

Start Button

The large size of the start button is designed to reflect its significance in the menu. It is on the right because my target audience will likely read from left to right, meaning that they will see the difficulty

slider first and respond to that before pressing play. The triangle inside the play button is designed as the general symbol for 'play', in music, video or games. Due to this being so universal, I will likely remove the label above it. This would further centre it and remove unnecessary detail.

Title

The title sits above everything else, making it the first thing seen. Its audacity is similar to its competitors, thus it should make the game better known. The font above is a placeholder font for what would either be a hand drawn title, or a better font.

Tests Per Stage

Stage 1 - Player Creation

Test ID	Test	Test Data	Outcome	Actual Outcome
1A	Is the player sprite displayed?	Run program	Player sprite should be displayed at the player's location.	
1B	Is the player able to move, corresponding to keyboard input?	Run program	The player moves upon keyboard input. This accurately covers all necessary directions.	
1C	Can the player attack?	Run program	The attack animation is displayed after the relevant keyboard input.	
1D	Does the player fall to the ground after jumping?	Run program	The player falls from the air after jumping, giving the impression of gravity.	
1E	Does the player have values for `health`, `stamina` and `knockback`?	Run program	Yes, with relevant methods to get this data.	

Stage 2 - Enemy Creation

A class called `Enemy` would have been created in order for above stages. This stage populates that class with data, actions and methods.

Test ID	Test	Test Data	Outcome	Actual Outcome
2A	Is the enemy sprite displayed at their location?	Run program	The sprite is displayed wherever the enemy is.	
2B	Is the enemy able to move?	Run program	The enemy can move when commanded with keyboard	

Test ID	Test	Test Data	Outcome	Actual Outcome
			input.	
2C	Is the enemy able to move in response to data about the player?	Run program	The enemy uses player getter methods to decide when to move.	
2D	Can the enemy attack and decide when to through received data?	Run program	The enemy uses data such as player proximity to "decide" to attack.	
2E	Does the enemy have data for `health`, `stamina` and `knockback`?	Run program	Yes, with methods to return them.	

Stage 3 - Data Collection

Test ID	Test	Test Data	Outcome	Actual Outcome
3A	Do methods for returning data for character attributes such as `Health` and `Stamina` return accurate information?	Run program	The methods used to find and return data are accurate.	
3B	Is the method to calculate and return the distance between the player and enemy accurate?	Run program	The method is accurate, returning data in a form which is usable.	

Stage 4 - HUD

Test ID	Test	Test Data	Outcome	Actual Outcome
4A	Does the HUD displays accurate data for the health of either character?	Run program	The HUD displays coloured bars which display either characters' health in real time.	
4B	Does the HUD display a stamina bar for either character, updated in real time?	Run program	The HUD displays coloured bars which display either characters' stamina in real time.	
4C	Does the HUD dynamically display the player's cooldown as a bar?	Run program	The HUD displays the player's cooldown value, in real time, showing only it when necessary (cooldown is not zero).	

Test ID	Test	Test Data	Outcome	Actual Outcome
4D	Does the HUD display a play/pause button is the top left corner for the user?	Run program	A play/pause button is displayed in the appropriate region. On clicking it the icon/sprite changes to reflect the state, with the game being paused/resumed.	
4E	Do all of the bars show relevant information which is consistently updated?	Run program	Yes.	

Stage 5 - Start Menu

Test ID	Test	Test Data	Outcome	Actual Outcome
5A	Does the game start with a menu?	Run program	The game loads with a menu with the option to start the game.	
5B	Does this start menu include a difficulty slider?	Run program	Yes.	
5C	Does the difficulty slider accurately update a value which affects the difficulty of the game?	Run program	Yes.	
5D	Does the start button start the game?	Run program	On click, the start button begins the game with both the player and enemy set up?	

Stage 6 - Better Attacks

Test ID	Test	Test Data	Outcome	Actual Outcome
6A	Do attacks have a value for knockback as well as damage?	Run program	Yes.	
6B	Is knockback applied accurately to either character when necessary?	Run program	Yes.	

Stage 7 - Condition Checks

Test ID	Test	Test Data	Outcome	Actual Outcome
7A	Does the player win when the enemy's health is 0?	Run program	The player wins the game and an appropriate screen is	

Test ID	Test	Test Data	Outcome	Actual Outcome
			displayed to convey this.	
7B	Does the player lose the game when their health is 0?	Run program	The enemy wins and a message is shown to display this.	
7C	Can the player attack when their stamina is 0?	Run program	The player cannot attack until their stamina is sufficiently regenerated.	
7D	Does the stamina amount regenerate exponentially after a period of inactivity?	Run program	Yes.	
7E	Can the player use an attack when it is on `Cooldown`?	Run program	No, they must wait until the `Cooldown` timer has ended.	
7F	Is a character removed when their health is 0?	Run program	The character is removed from the scene.	

Stage 8 - Better Enemies

Test ID	Test	Test Data	Outcome	Actual Outcome
8A	Does the enemy load with random strength and knockback in accordance with `difficulty`?	Run program	Yes.	
8B	Does the enemy have a random size in accordance with `difficulty`?	Run program	Yes.	

Final Tests

Test ID	Test	Test Data	Purpose	Actual Outcome
F1	Player Enemy collision	Run program	Tests if the player and enemy are capable of colliding.	
F2	Appropriate knockback levels for attacks	Run program	Tests if the level of knockback applied is suitable for player and enemy.	
F3	Appropriate health values in correspondence with damage	Run program	Tests how long the average game could last and is this is consistent with my analysis and planning.	
F4	The player dying ends the game	Run program	To check if there is an appropriate response to inform the player that they have lost.	

Test ID	Test	Test Data	Purpose	Actual Outcome
F5	The start screen starts the game correctly	Run program	To ensure that random values for the enemy are correctly created with reference to the difficulty slider.	
F6	The gravity of the game works to achieve a pseudo realism	Run program	To see how the game feels to play for a player. A lower gravity could make for a slower game, with more time spent in the air.	

Post Development Questions

Once each of my stages have been implemented, and I have alpha tested, I would like to ask some of my friends and peers to beta test my game. In order for this to be effective and a good use of time, I will put together series of questions to ask them once they have played the game on each difficulty setting.

My questions should aim to cover information about the following:

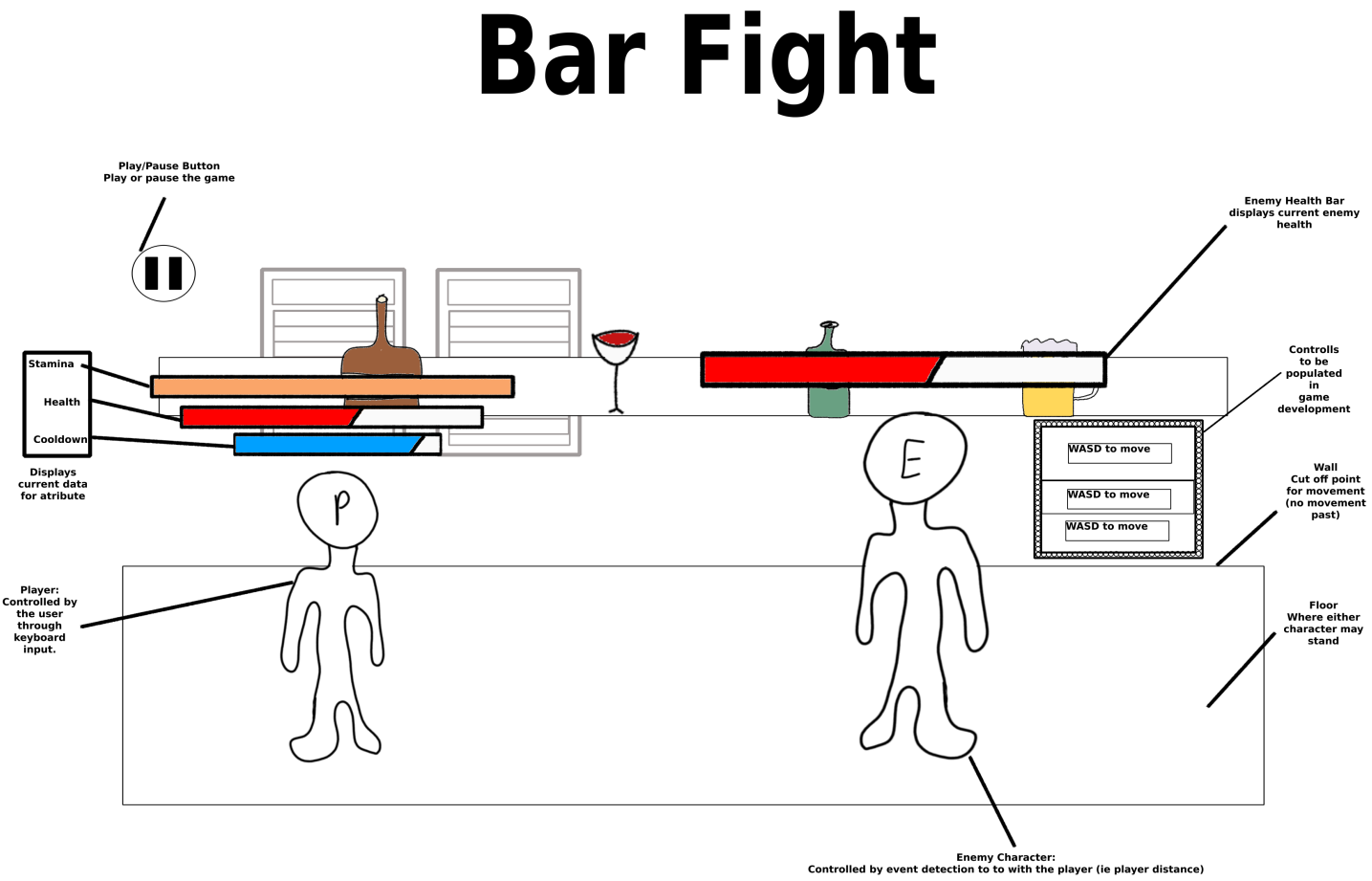
- User friendliness
- Intuitive design
- Likeability of the design
- Difficulty of each setting
- How long each difficulty took them
- The usability of the controls
- Did the game perform well (high frames per second, etc)
  In light of this, my questions will probably look something like this:

1. On a scale of 1 to 10, how user friendly is the game?
2. On a scale of 1 to 10, how intuitive would you describe the design?
3. Did you like the design?
4. On a scale of 1 to 10, how hard was the hardest setting?
5. On a scale of 1 to 10, how hard was the default setting?
6. On a scale of 1 to 10, how hard was the easiest setting?
7. How long did the hardest difficulty take you?
8. How long did the default difficulty take you?
9. How long did the easiest difficulty take you?
10. Did you find the controls to be easy to learn and use?
11. Would you say the game performed well on your system?
12. Did you encounter any bugs/glitches?
    My hope is that the beta testers will enjoy the game and give constructive feedback to work on before it is finished.

This questionnaire will most likely be sent via Microsoft Forms, due to the Microsoft Office Suite being available to my college. This would make it the easiest form of communication with integrated data graphics.

GUI Design Justifications

Main Screen



The above is a rough outline of what I hope my project might become. As of this, details such as non-important colours including background and player sprites are not shown above.
This screen is the most detailed, because it will be displayed the majority of the time.

Bars

Above each will be coloured bars to display data about metrics corresponding to each character. The amount of the bar coloured in would be the percentage of the metric from full, for example if the player had 40 health remaining down from 100, the health bar would be 40% full.

Colours

The colours for each bar have been chosen to match the aesthetic of my game, whilst taking inspiration from other games of similar genres or categories. As such health would be red; stamina

tan/peach; cooldown blue.

Visibility

As is above, more data would be available for the player's character than for the enemy. This adds more challenge to the game as well as authenticity with the player not knowing some data about their opponent such as stamina.
Furthermore, the cooldown bar would only be temporarily displayed. This being when it is needed (not 0). This would increase the player's view by only showing data useful at that moment.

Size Variance

The larger the value for a character's maximum or starting health, the larger the bar for that should be. In practice this would affect only the enemy once the relevant development cycle is complete.

Play/Pause Button

This button will sit in the top left corner, as my market research has led me to conclude is standard, due to the space freed up on the screen. Upon clicking, it will pause the game and change its sprite to reflect the state. In practice, pausing will look like stopping the movement of either player or enemy and halting the regeneration of stamina or cooldown.

Characters

The game will feature two characters, being the player and the enemy, both visible on the screen. Their movement will be shown live, making the game playable. I plan to draw sprites for each, matching them to the overall look of the game. The examples shown above are rough outlines of what I hope mine become. The size difference between the player and enemy character is intended, and hopefully will reflect plays with a higher difficulty, where the enemy is larger than the player.

Controls Screen

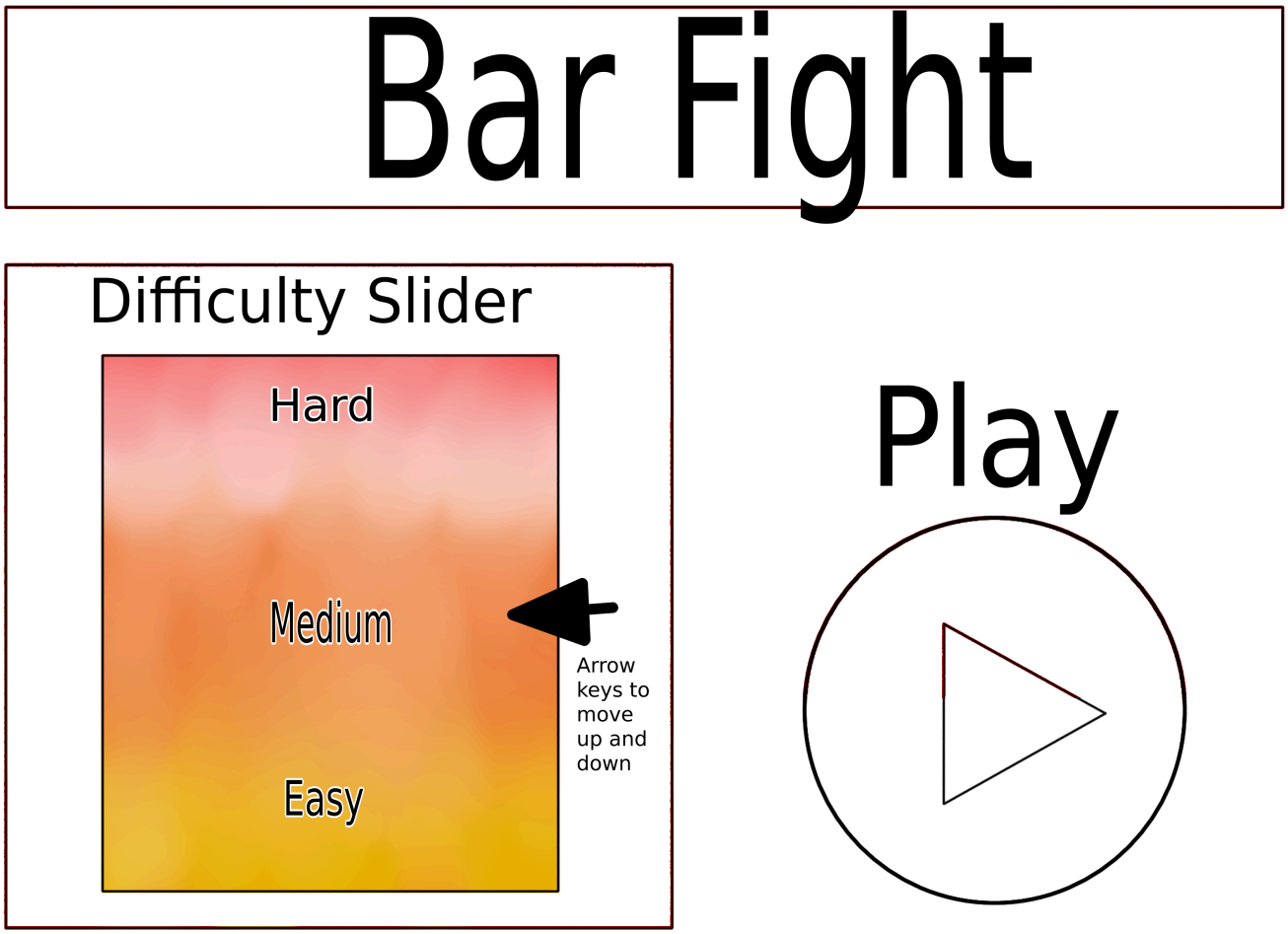
In the background of the scene will be a small display in the style of a bar's menu. It will have on it the various controls to perform basic actions such as move, punch, etc. I have chosen this style because it will reflect the general style of the game. This will increase the continuity of the game, making it feel better tuned and stylised.

Background

The background in the image above is a rough simplification. I plan to keep elements like the bottles, shelves and drinks, but with a more full scene. The scene created by it should feel dark and quite

moody, like a small pub.

Start Menu



Like the main game GUI mock up, this is an abstraction of what I hope the actual thing looks like. Some features above are exaggerated so that they are quicker to find, and more obvious to make. Furthermore, there is no background image, hence the white background.
I have this main menu in order to bridge the gap between loading the game and fighting. This allows the player to spend time selecting a difficulty and preparing themselves.

Difficulty Slider

The difficulty slider above is coloured based on its difficulty. The cursor/selector would be set at medium by default, making the colour system intuitive to the user. The idea of moving the selector with arrow keys may be a little confusing, or not explicitly obvious, so I would like to have a label for this information. I also have deliberately put it on the left, such that the start screen is dominated by itself and the play buttons - the two main features of my start screen. As well as this, the position of the difficulty slider echoes games of a similar category, and the colour scheme would demonstrate this further. The result is a dynamic looking menu, with a clear colour scheme which evidences itself to its genre.

Start Button

The large size of the start button is designed to reflect its significance in the menu. It is on the right because my target audience will likely read from left to right, meaning that they will see the difficulty slider first and respond to that before pressing play. The triangle inside the play button is designed as the general symbol for 'play', in music, video or games. Due to this being so universal, I will likely remove the label above it. This would further centre it and remove unnecessary detail.

Title

The title sits above everything else, making it the first thing seen. Its audacity is similar to its competitors, thus it should make the game better known. The font above is a placeholder font for what would either be a hand drawn title, or a better font.

Tests

Tests Per Stage

Stage 1 - Player Creation

Test ID	Test	Test Data	Outcome	Actual Outcome
1A	Is the player sprite displayed?	Run program	Player sprite should be displayed at the player's location.	
1B	Is the player able to move, corresponding to keyboard input?	Run program	The player moves upon keyboard input. This accurately covers all necessary directions.	
1C	Can the player attack?	Run program	The attack animation is displayed after the relevant keyboard input.	
1D	Does the player fall to the ground after jumping?	Run program	The player falls from the air after jumping, giving the impression of gravity.	
1E	Does the player have values for `health`, `stamina` and `knockback`?	Run program	Yes, with relevant methods to get this data.	

Stage 2 - Enemy Creation

A class called `Enemy` would have been created in order for above stages. This stage populates that class with data, actions and methods.

Test ID	Test	Test Data	Outcome	Actual Outcome
2A	Is the enemy sprite displayed at their location?	Run program	The sprite is displayed wherever the enemy is.	
2B	Is the enemy able to move?	Run program	The enemy can move when commanded with keyboard input.	
2C	Is the enemy able to move in response to data about the player?	Run program	The enemy uses the getter methods from stage 2 to decide when to move.	
2D	Can the enemy attack and decide when to through received data?	Run program	The enemy uses data such as player proximity to "decide" to attack.	
2E	Does the enemy have data for `health`, `stamina` and `knockback`?	Run program	Yes, with methods to return them.	

Stage 3 - Data Collection

Test ID	Test	Test Data	Outcome	Actual Outcome
3A	Do methods for returning data for character attributes such as `Health` and `Stamina` return accurate information?	Run program	The methods used to find and return data are accurate.	
3B	Is the method to calculate and return the distance between the player and enemy accurate?	Run program	The method is accurate, returning data in a form which is usable.	

Stage 4 - HUD

Test ID	Test	Test Data	Outcome	Actual Outcome
4A	Does the HUD displays accurate data for the health of either character?	Run program	The HUD displays coloured bars which display either characters' health in real time.	
4B	Does the HUD display a stamina bar for either character, updated in real time?	Run program	The HUD displays coloured bars which display either characters' stamina in real time.	

Test ID	Test	Test Data	Outcome	Actual Outcome
4C	Does the HUD dynamically display the player's cooldown as a bar?	Run program	The HUD displays the player's cooldown value, in real time, showing only it when necessary (cooldown is not zero).	
4D	Does the HUD display a play/pause button is the top left corner for the user?	Run program	A play/pause button is displayed in the appropriate region. On clicking it the icon/sprite changes to reflect the state, with the game being paused/resumed.	
4E	Do all of the bars show relevant information which is consistently updated?	Run program	Yes.	

Stage 5 - Start Menu

Test ID	Test	Test Data	Outcome	Actual Outcome
5A	Does the game start with a menu?	Run program	The game loads with a menu with the option to start the game.	
5B	Does this start menu include a difficulty slider?	Run program	Yes.	
5C	Does the difficulty slider accurately update a value which affects the difficulty of the game?	Run program	Yes.	
5D	Does the start button start the game?	Run program	On click, the start button begins the game with both the player and enemy set up?	

Stage 6 - Better Attacks

Test ID	Test	Test Data	Outcome	Actual Outcome
6A	Do attacks have a value for knockback as well as damage?	Run program	Yes.	
6B	Is knockback applied accurately to either character when necessary?	Run program	Yes.	

Stage 7 - Condition Checks

Test ID	Test	Test Data	Outcome	Actual Outcome
7A	Does the player win when the enemy's health is 0?	Run program	The player wins the game and an appropriate screen is displayed to convey this.	
7B	Does the player lose the game when their health is 0?	Run program	The enemy wins and a message is shown to display this.	
7C	Can the player attack when their stamina is 0?	Run program	The player cannot attack until their stamina is sufficiently regenerated.	
7D	Does the stamina amount regenerate exponentially after a period of inactivity?	Run program	Yes.	
7E	Can the player use an attack when it is on `Cooldown`?	Run program	No, they must wait until the `Cooldown` timer has ended.	
7F	Is a character removed when their health is 0?	Run program	The character is removed from the scene.	

Stage 8 - Better Enemies

Test ID	Test	Test Data	Outcome	Actual Outcome
8A	Does the enemy load with random strength and knockback in accordance with `difficulty`?	Run program	Yes.	
8B	Does the enemy have a random size in accordance with `difficulty`?	Run program	Yes.	

Final Tests

Test ID	Test	Test Data	Purpose	Actual Outcome
F1	Player Enemy collision	Run program	Tests if the player and enemy are capable of colliding.	
F2	Appropriate knockback levels for attacks	Run program	Tests if the level of knockback applied is suitable for player and enemy.	
F3	Appropriate health values in correspondence with damage	Run program	Tests how long the average game could last and is this is consistent with my analysis and planning.	
F4	The player dying ends the game	Run program	To check if there is an appropriate response to inform the player that	

Test ID	Test	Test Data	Purpose	Actual Outcome
			they have lost.	
F5	The start screen starts the game correctly	Run program	To ensure that random values for the enemy are correctly created with reference to the difficulty slider.	
F6	The gravity of the game works to achieve a pseudo realism	Run program	To see how the game feels to play for a player. A lower gravity could make for a slower game, with more time spent in the air.	

Post Development Plan

Once each of my stages have been implemented, and I have alpha tested, I would like to ask some of my friends and peers to beta test my game. In order for this to be effective and a good use of time, I will put together series of questions to ask them once they have played the game on each difficulty setting.

My questions should aim to cover information about the following:

- User friendliness
- Intuitive design
- Likeability of the design
- Difficulty of each setting
- How long each difficulty took them
- The usability of the controls
- Did the game perform well (high frames per second, etc)
  In light of this, my questions will probably look something like this:

1. On a scale of 1 to 10, how user friendly is the game?
2. On a scale of 1 to 10, how intuitive would you describe the design?
3. Did you like the design?
4. On a scale of 1 to 10, how hard was the hardest setting?
5. On a scale of 1 to 10, how hard was the default setting?
6. On a scale of 1 to 10, how hard was the easiest setting?
7. How long did the hardest difficulty take you?
8. How long did the default difficulty take you?
9. How long did the easiest difficulty take you?
10. Did you find the controls to be easy to learn and use?
11. Would you say the game performed well on your system?
12. Did you encounter any bugs/glitches?
    My hope is that the beta testers will enjoy the game and give constructive feedback to work on before it is finished.
    This questionnaire will most likely be sent via Microsoft Forms, due to the Microsoft Office Suite

being available to my college. This would make it the easiest form of communication with integrated data graphics.

Development

Stage 1

Description and Goals

Stage 1 involves creating the player with adequate movement abilities. This extends to attacks, which for now just involves changing the sprite.

To Do List

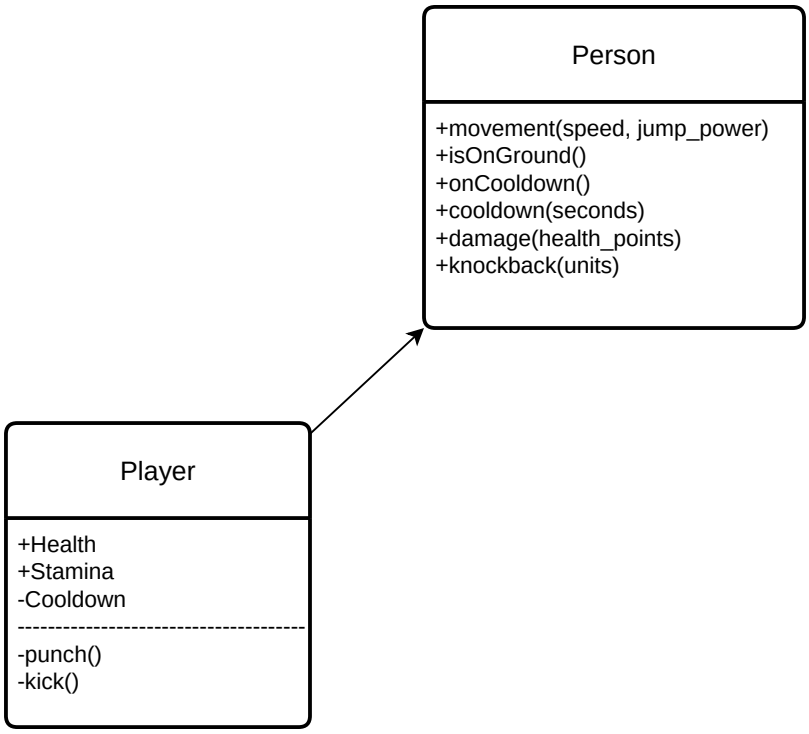
Features

- [x] ~~Player~~
  - [x] ~~Can move with keyboard input (left, right, jump)~~
  - [x] ~~Can attack with keyboard input~~
  - [x] ~~Player returns to ground after jumping~~

Tests

- [x] ~~Player sprite displayed?~~
- [x] ~~Accurate movement?~~
- [x] ~~Attack animation displayed?~~
- [x] ~~Gravity?~~
- [x] ~~Health Values?~~

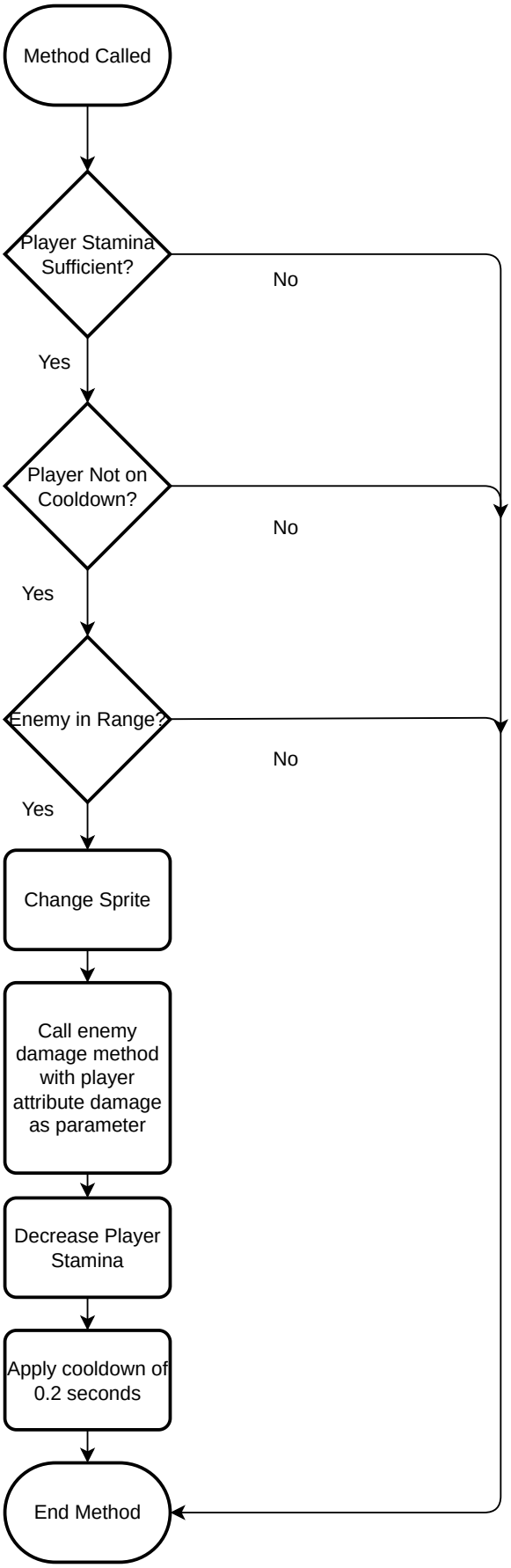
Class Diagram



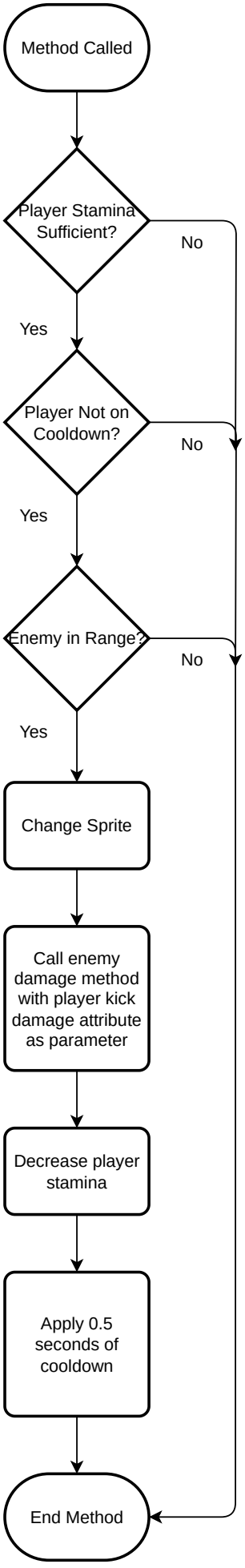
This class diagram holds all of the attributes and methods for my player. I have produced this because it will be incredibly useful to know all of the methods which I will need to implement and attributes to declare. Please note that this would not include all variables. Ideally, I would have some declared like constants to make it easier to modify data such as horizontal acceleration.

Methods

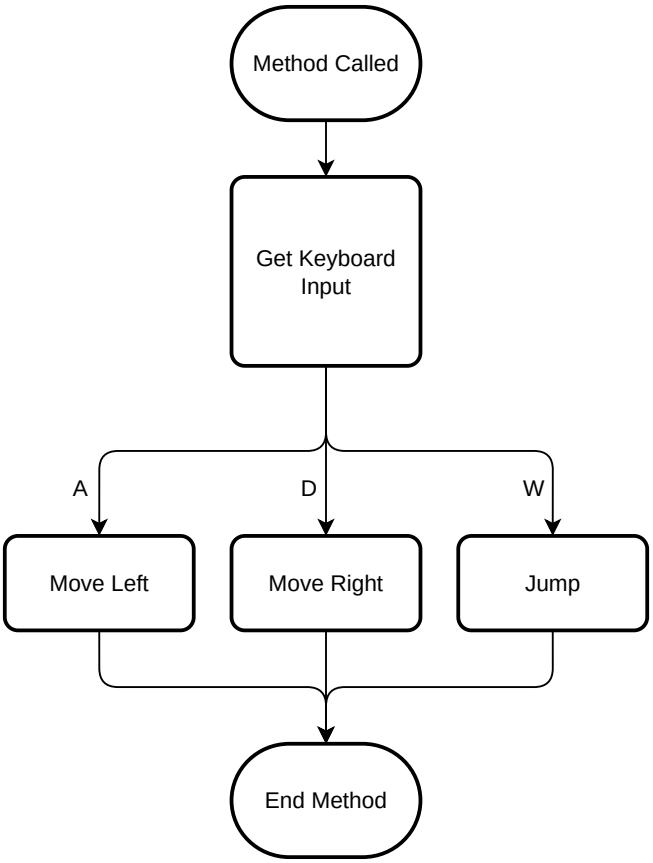
punch()


Method Called
Player Stamina Sufficient?
No
Yes
Player Not on Cooldown?
No
Yes
Enemy in Range?
No
Yes
Change Sprite
Call enemy damage method with player attribute damage as parameter
Decrease Player Stamina
Apply cooldown of 0.2 seconds
End Method

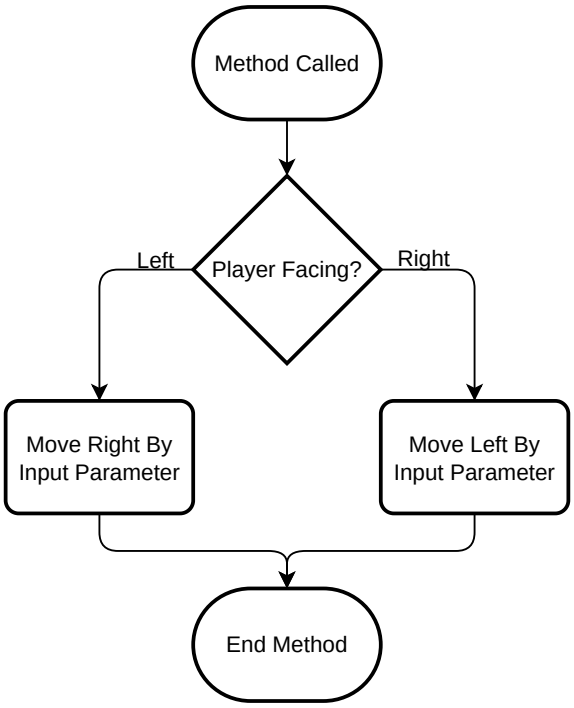
kick()


Method Called
Player Stamina Sufficient?
No
Yes
Player Not on Cooldown?
No
Yes
Enemy in Range?
No
Yes
Change Sprite
Call enemy damage method with player kick damage attribute as parameter
Decrease player stamina
Apply 0.5 seconds of cooldown
End Method

movement()



knockback()



damage()

```
function damage(int hp) then
     this.health = this.health - hp;
     end function
```

Attributes

Health

Attribute to control the character's health. When `player.health == 0`, game over.

Stamina

Controls whether or not the player is allowed to attack.

Cooldown

Same as `stamina` but with more direct consequences. Stamina is designed to limit and restrict against attacking relentlessly, whilst `cooldown` should stop the player from being able to attack every frame (too fast).

Notes

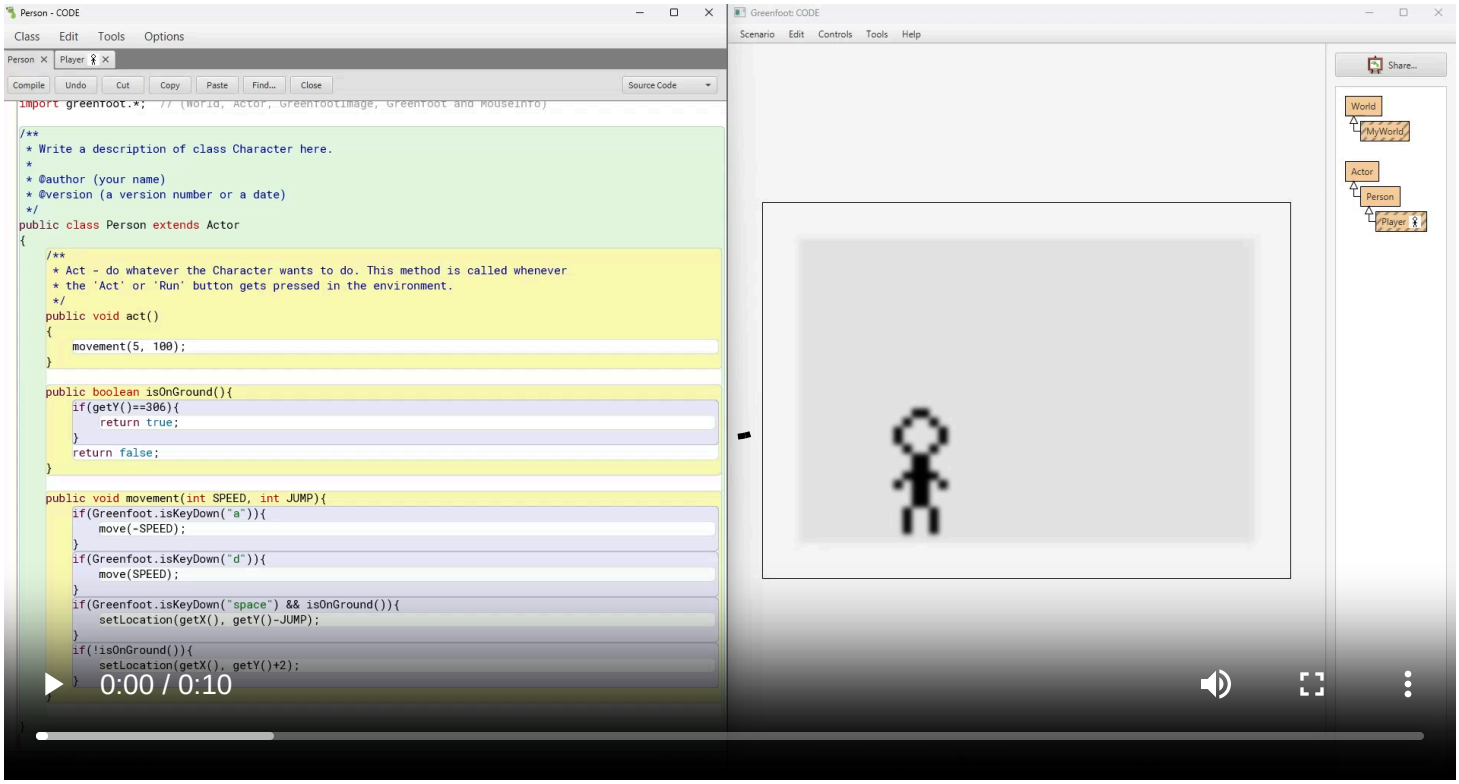
references > How to round to an integer

Pixel Rounding Error



This is the first major error which I encountered. I was overambitious in my designing of a jump method, and wanted it to be physically accurate, using equations of motion. The problem with this was that Greenfoot would not have been able to move an object a fractional number of pixels, meaning that my `JUMP_VELOCITY_INIT` and `GRAVITY` variables would not be viable, even if I were to round them. I tried to solve this by multiplying the number of pixels in the world by 100, in order to do the same to the variables and reduce the rounding errors. This would not work due to Greenfoot's own limit on pixels. This led to me having to scrap this idea.

Moving and Jumping Proof

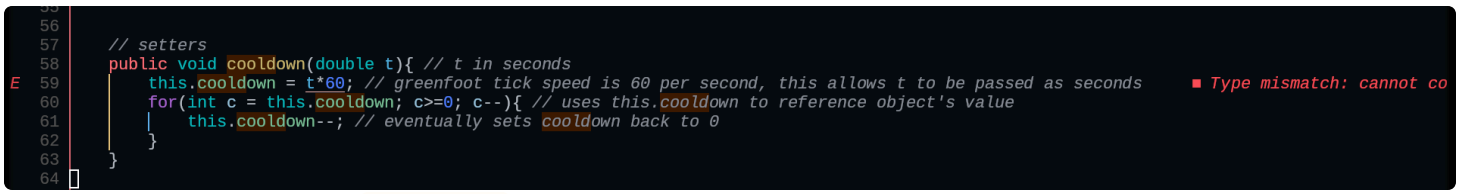


In this video (moving_and_jumping.mp4), we can see the player moving and jumping, through the inherited move method from the parent class `Person`. The jumping is quite janky because of the use of `setLocation(getX(), getY()-JUMP)`. I tried to incorporate a gradual increase such as

```
while(getY()>=FLOOR-JUMP){
     setLocation(getX(), getY()-5);
}
while(getY()<FLOOR){
     setLocation(getX(), getY()+5);
}
```

But this became a problem with Greenfoot. I am still unsure why, but it would cause the program to crash. The current solution is now a compromise.

Old Cooldown Method



This method was the old `cooldown` method before the new one
(

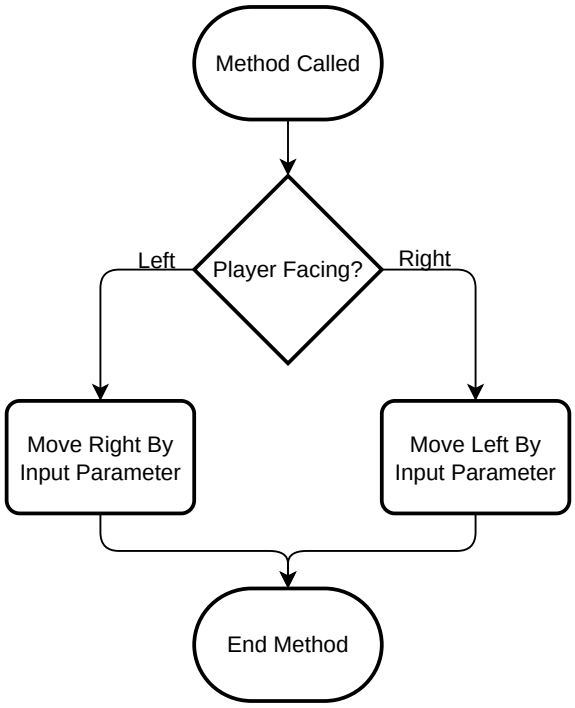
```
import java.math.RoundingMode;
import java.text.DecimalFormat;

public void cooldown(double t){
     double unrounded_cooldown = t*60;
     DecimalFormat df = new DecimalFormat("###"); // where unused digits are 0
     df.setRoundingMode(RoundingMode.CEILING); // rounds up from t*60
     this.cooldown = df.format(unrounded_cooldown); // sets the cooldown to
appropriate integer
     for(int c = this.cooldown; c>=0; c--){ // same logic as if it were an int
       this.cooldown--;
     }
}
```

). As can be seen, `this.cooldown` is incorrectly assigned as `t*60`. This is because I had forgotten my initial plan to have the punch cooldown as 0.2 seconds. This adjustment was the reason which I had to change to an algorithm with explicit rounding.

With regards to Knockback

I have at present chosen not to implement the full `Knockback` method as was planned in my flowchart.



This is because I think that finding out which way the player was facing (especially in Greenfoot), would take quite a lot of time. Therefore, I have at present left this either for the future or permanence. The current method stands at:

```
public void knockback(int k){
     move(-k);
}
```

This takes a heuristic approach, assuming that the player will always be on the left, attacking right. It is far more concise than the flow chart demands it be, which is why I have decided that it is currently the best option.

Fixing A Rounding Error

In the previous fix for the `cooldown` method, I employed the libraries `math.RoundingMode` and `text.DecimalFormat`. This is shown in Old Cooldown Method. I have since dealt with an error produced by the `df.format()'` call. As it happens, this method returns a string type, which caused a problem with `cooldown`, being defined as an integer. I fixed this by adding

```
Integer.parseInt(df.format(target));
```

thus converting the "string" to the desired integer.

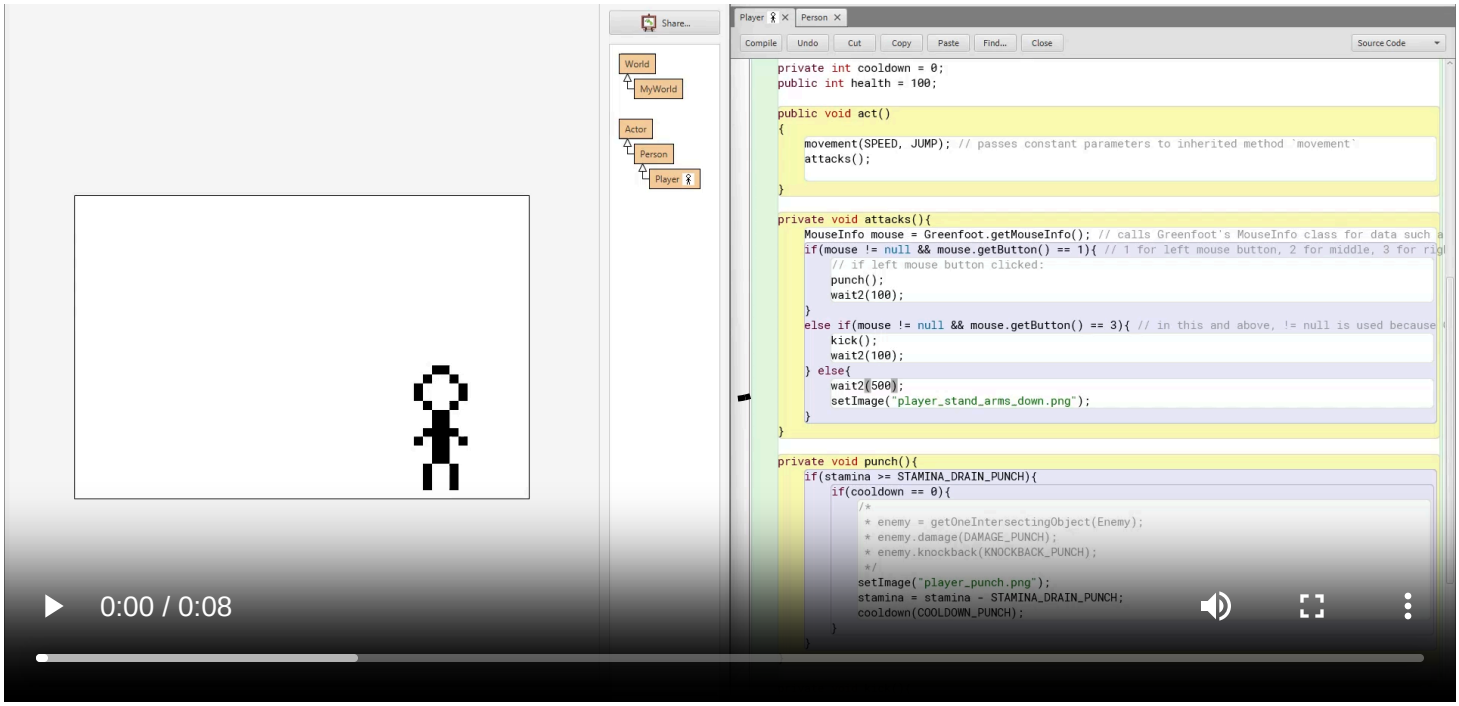
Round() Method

Whilst implementing a `wait2()` method (further explained later), I was going to reuse the rounding code from my `cooldown(double t)` method. Instead of doing this, I optimised the code by introducing a new `round(double target)` function.

```
public int round(double target){
     DecimalFormat df = new DecimalFormat('###');
     df.setRoundingMode(RoundingMode.CEILING);
     return Integer.parseInt(df.format(target));
}
```

Sprite Setting

Whilst I had technically completed my test plan, I had not accounted for some of the problems which I may have encountered. One of them is that I had not checked that the player's sprite would revert back to the default after an attack sprite is displayed.



(attack_sprite_timing_error.mp4)

```
private void attacks(){
    MouseInfo mouse = Greenfoot.getMouseInfo(); // calls Greenfoot's Mousel
    if(mouse != null && mouse.getButton() == 1){ // 1 for left mouse buttor
        // if left mouse button clicked:
        punch();
        wait2(100);
        setImage("player_stand_arms_down.png");
    }
    if(mouse != null && mouse.getButton() == 3){ // in this and above, != r
        kick();
        wait2(1);
        setImage("player_stand_arms_down.png");
    }
}
```

I thought that it would be an easy fix like in the above image. Sadly this does not work. I was able to trace the point of vulnerability to the `wait2()` which I then tried to fix.

```
public void wait2(double secs){     // named as such because wait() is al
    int ticks = round(secs*6000, "###"); // rounds value for how many gre
    int ticks_remaining = ticks+831308756; // copy of ticks to iterate dc
    for(int i=0; i<=ticks; i++){ // ticks used here because ticks_remain
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}
```

As you can see in

```
int ticks_remaining = ticks+831308756;
```

I tested the function by iterating through arbitrarily large numbers. This is because the core idea of the function is that if each second runs 60 frames, then 60 frames can be populated with a 'spinner loop' to accurately wait 1 second. My test would have accounted for a minimum of 13855145.93333333... seconds. Unfortunately this did not work so I theorised that the error stemmed from the fact that it was an inherited method and so might be running in a different thread.

Player | Person

Compile | Undo | Cut | Copy | Paste | Find... | Close

```
final double COOLDOWN_KICK = 0;//0.5;
final int STAMINA_DRAIN_KICK = 0;//20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 1000;
private int cooldown = 0;
public int health = 100;


public void act()
{
    movement(SPEED, JUMP); // passes constant para
    attacks();

}

public void wait2(double secs){     // named as su
    int ticks = round(secs*6000, "###"); // rounds
    int ticks_remaining = ticks+831308756; // copy
    for(int i=0; i<=ticks; i++){ // ticks used her
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}
```

Unfortunately this was to the same effect, even with slight modification as below:

```
public void wait2(int secs){     // named as such because wait() is already
    //int ticks = round(secs*6000, "###"); // rounds value for how many gree
    int ticks_remaining = 831308756; // copy of ticks to iterate down
    /*for(int i=0; i<=secs*60; i++){ // ticks used here because ticks_remair
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }*/
    while(ticks_remaining > 0){
        ticks_remaining--;
    }
}
```

Fixing the Sprite Setting

Today (01.10.26) I fixed the sprite setting issue. I realised that it was caused by the fact that it was the **main act method** which called 60 times per second. This in conjunction with my broken `wait2()` function led to me implementing a manual timer held in the Player act method.

Player | Person

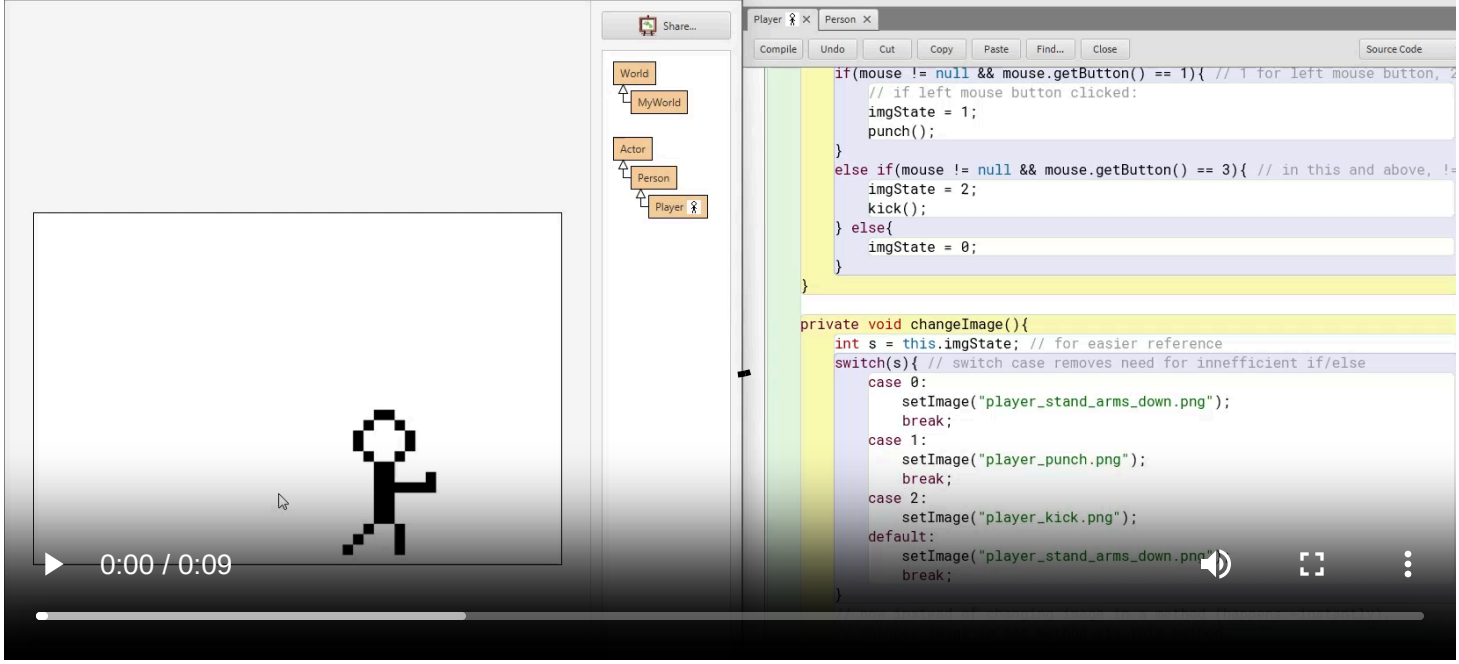
Compile | Undo | Cut | Copy | Paste | Find... | Close | Source C

```
final int STAMINA_DRAIN_PUNCH = 10;
final double COOLDOWN_PUNCH = 0.2;
final double COOLDOWN_KICK = 0.5;
final int STAMINA_DRAIN_KICK = 20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 100;
private int cooldown = 0;
public int health = 100;
private int imgState = 0; // 0 as default, 1 for punch, 2 for kick
private int imgCountdown = 0; /* theoretically should limit time for ima
change. Each act method this will increment and thus change the image fo
sprite.
 */
public void act()
{
    // variables
    imgCountdown++;
    if(cooldown > 0){
        cooldown--;
    }
    if(stamina < STAMINA_MAX){
        stamina++;
    }
    // image changes
    if(imgCountdown == 30){
        changeImage();
        imgCountdown = 0;
    }
```

The fact that this solution works is making me consider a solution to the initial wait problem. As a result, in my next stage I might include a similar wait method with the decrementing number is stored in a super class outside of the method. It could be added to with the act method responsible for decreasing the cooldown value once per tick.

Proof:



(fixed_sprite_changes.mp4)

Stage 2

Description and Goals

Stage 2 plans to implement the enemy to the game, allowing further stages with it involved. It will take a large quantity of player methods.

To Do List

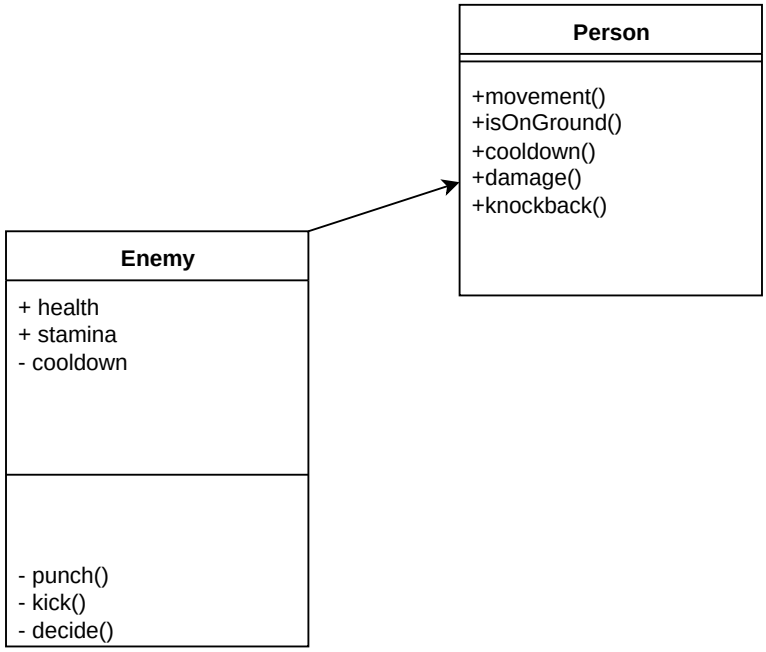
Features

- [x] ~~Create enemy class~~
- [x] ~~Enemy has appropriate attributes~~
- [x] ~~Enemy movement (with keyboard)~~
- [x] ~~Enemy attacks (with keyboard)~~

Tests

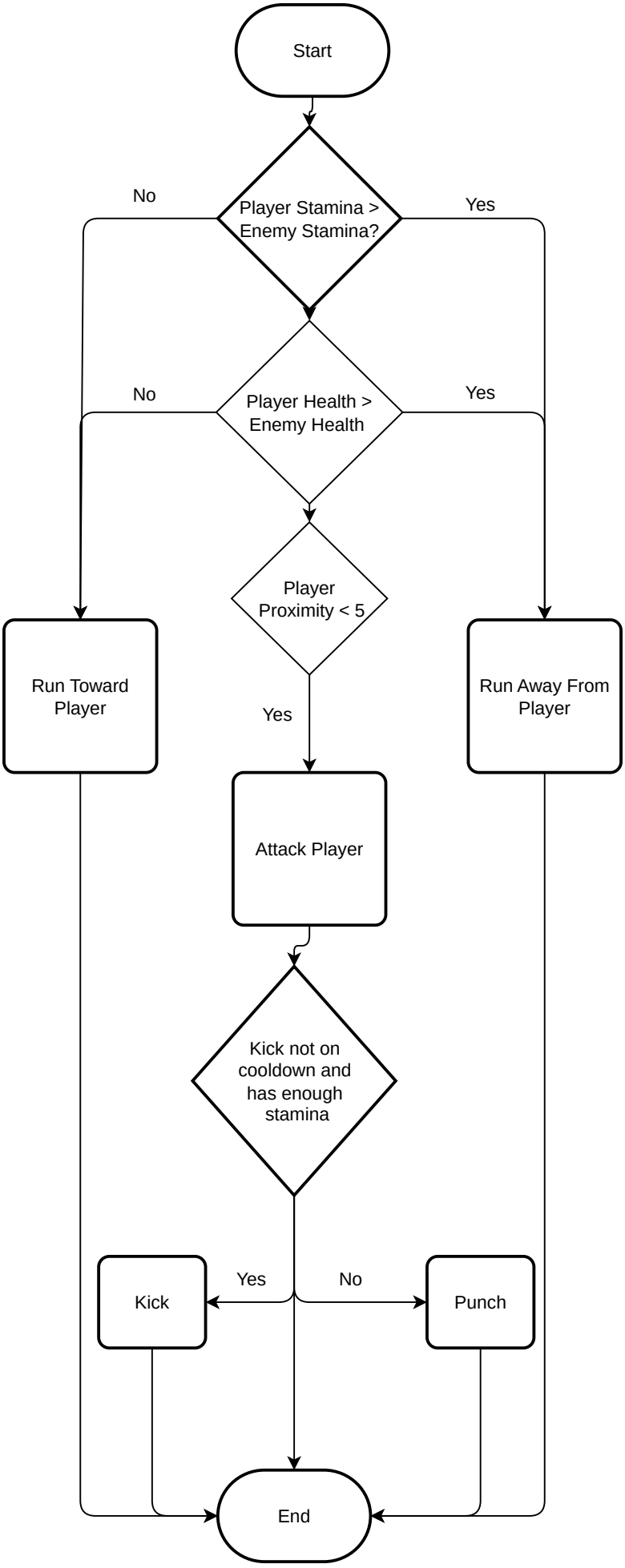
- [x] ~~Enemy can move~~
- [x] ~~Enemy can attack~~
- [x] ~~Enemy can take damage~~
- [x] ~~Enemy can give damage~~
- [x] ~~Enemy can be knocked back~~
- [x] ~~Enemy can knock back~~

Class Diagram

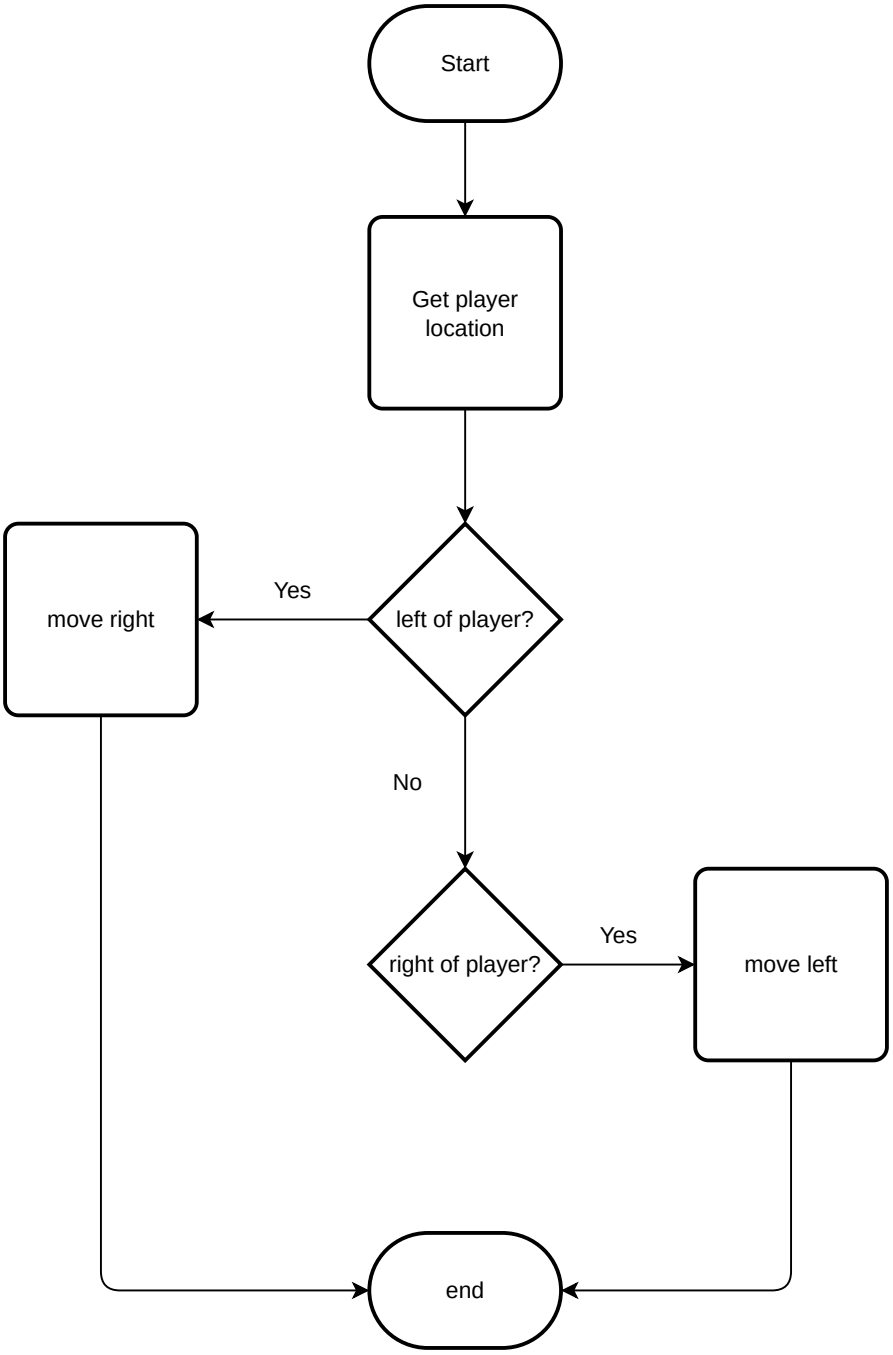


Methods

Decide


Start
Player Stamina > Enemy Stamina?
No
Yes
Player Health > Enemy Health
No
Yes
Player Proximity < 5
Run Toward Player
Run Away From Player
Yes
Attack Player
Kick not on cooldown and has enough stamina
Kick
Yes
No
Punch
End

runToward()



Attributes

Identical to player with modification in the constructor with a scalar modifier which will allow a difficulty slider to be implemented.

Notes

Null Pointer Errors

Due to this character's creation and actions hinging on the `player` class, for detection and attribute initiation, I had to reference this class a lot. This became challenging after receiving quite a few `null`

`pointer exception`'s. I ended up fixing this by passing the created player into `enemy`'s constructor in the `prepare()` method in the `World` class.

```
private void prepare()
    {

        Player player = new Player();
        addObject(player,PlayerStartX,PlayerStartY);
        // int SPEED, int JUMP, int SM, int DM, int KM
        Enemy enemy = new Enemy(5, 200, 100, 2, 3, player);
        addObject(enemy,395,306);
    }
```

With the enemy constructor looking like this:

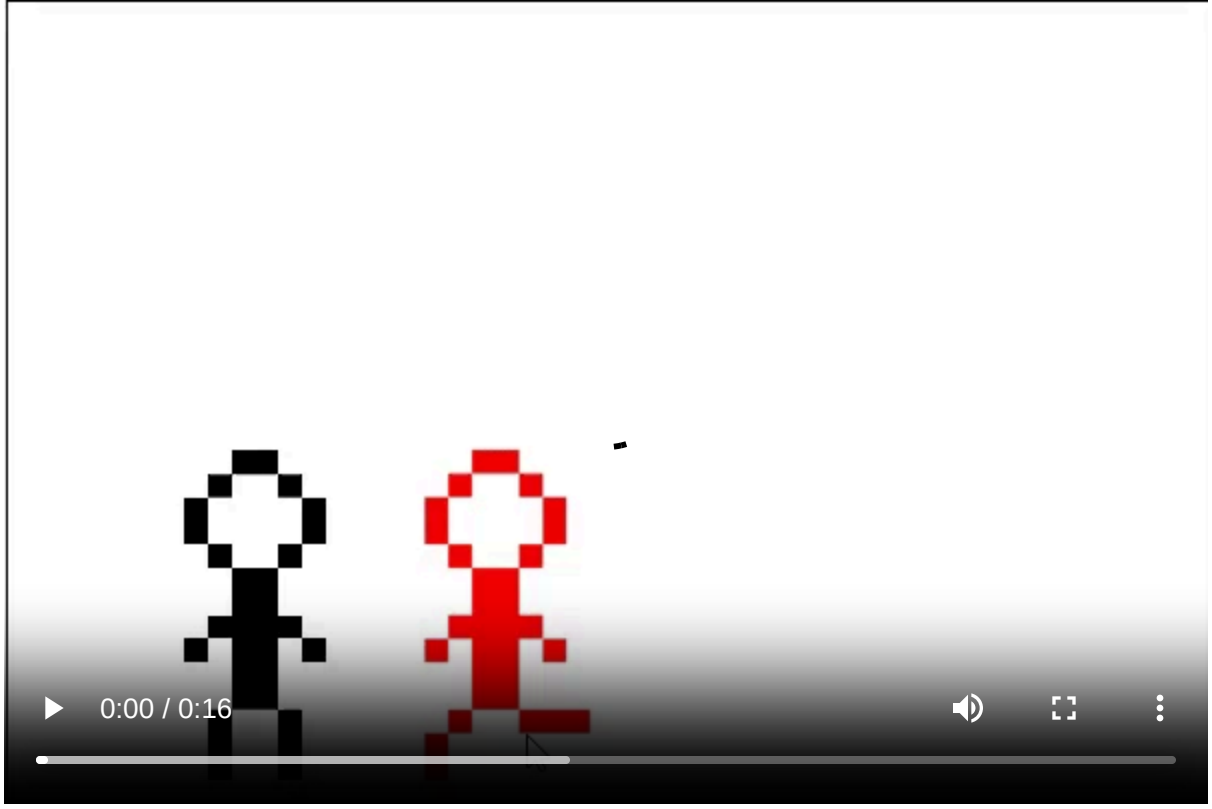
```
public Enemy(int SPEED, int JUMP, int SM, int DM, int KM, Player nemesis){
        this.SPEED = SPEED;
        this.JUMP = JUMP;
        this.STAMINA_MAX = SM;
        this.p = nemesis;
        // DM stands for Damage Multiplier and takes 1 as player attributes

        this.DAMAGE_PUNCH = round((Player.DAMAGE_PUNCH * DM), "###");
        this.DAMAGE_KICK = round((Player.DAMAGE_KICK * DM), "###");
        // KM stands for knockback multiplier and takes from player
        this.KNOCKBACK_PUNCH = round((Player.KNOCKBACK_PUNCH * DM), "###");
        this.KNOCKBACK_KICK = round((Player.KNOCKBACK_KICK * DM), "###");

    }
```

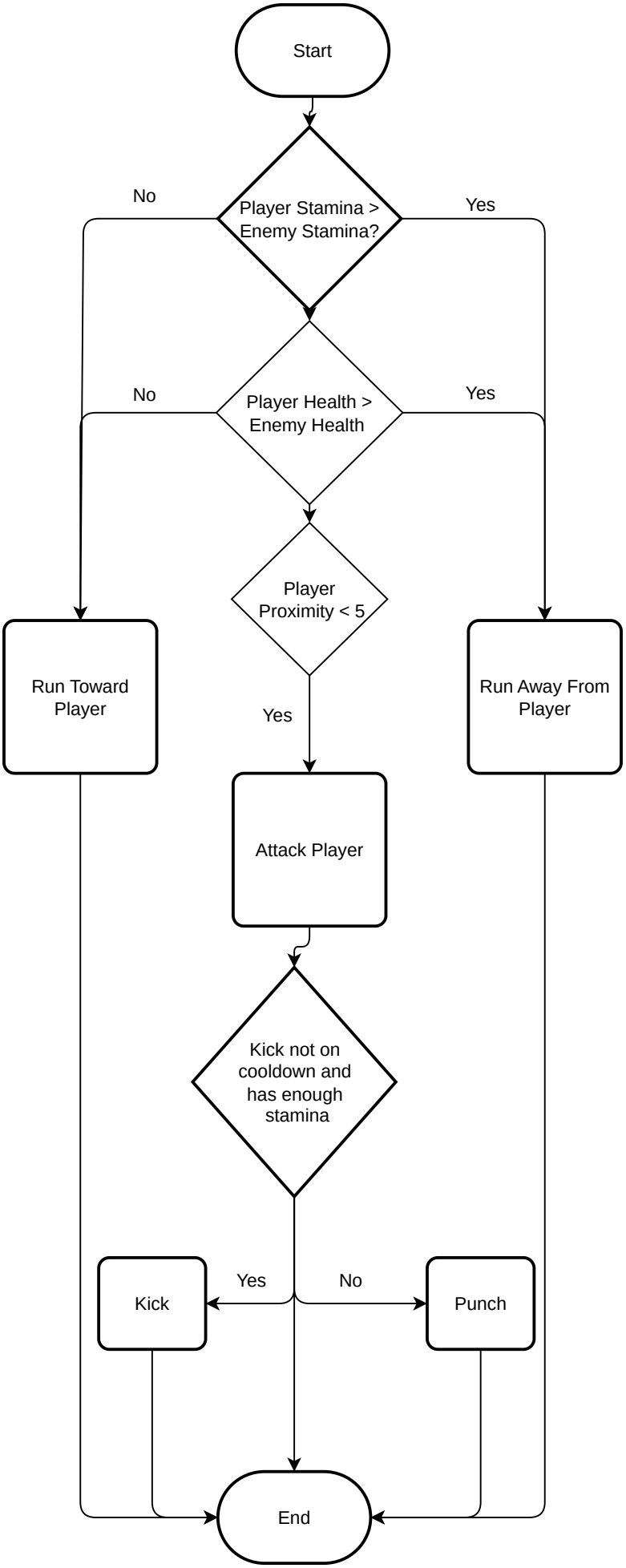
By passing in the player and storing it as a private global attribute, I could always reference the desired object from anywhere in the code. Because of this I decided that using a global variable to store this object would be the most appropriate way to avoid null pointer exceptions.

Decision Based Fighting



In this video (fighting.mp4), the enemy, in red, is attacking the player, black. This is achieved by the checking of conditions as outlined above

Decide


Start
Player Stamina > Enemy Stamina?
No
Yes
Player Health > Enemy Health
No
Yes
Player Proximity < 5
Run Toward Player
Run Away From Player
Yes
Attack Player
Kick not on cooldown and has enough stamina
Kick
Yes
No
Punch
End

The issue with some of this is that I did not foresee the repetition of the basic `punch` attack after stamina is below a threshold. This happens because the enemy is constantly checking if its stamina is high enough to kick, which it is not due to its barrage of attacks. I am hoping that this is rectified in future with a fix to the cooldown system.

Stage 3

Description and Goals

This stage aims to fully flesh out the data collection system. In planning, this would have been the final part of creating a functioning enemy, wiring it up to decisions. However a lot of this was covered in my Stage 2, when I decided that it would have been easier to implement the decision method at that point in time for purposes such as testing and experimentation. Consequently, this stage has been made redundant.

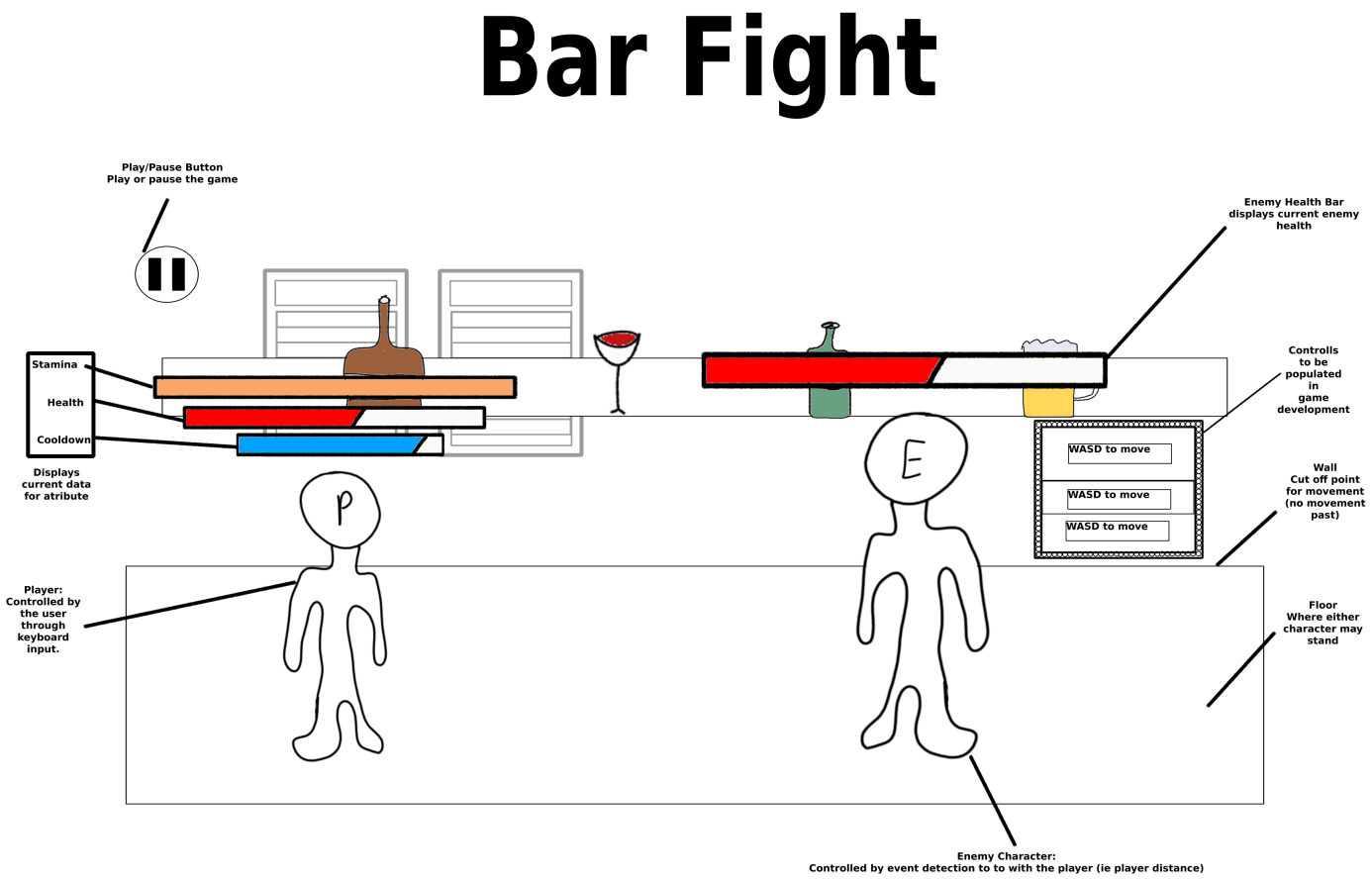
Stage 4

Description and Goals

This stage is responsible for creating the HUD (Heads Up Display) for the game. This will consist of the following elements:

- Player health bar
- Enemy health bar
- Player stamina bar
- Player cooldown bar (shown only when necessary)
- Pause/Play button
  These are designed to make the experience of playing the game easier because the person playing the game will be able to properly visualise the value of their health. This will allow them to

make informed decisions about attacking. The result should resemble the image below:



To Do List

Features

- [x] ~~Player Health Bar~~
- [x] ~~Player Stamina Bar~~
- [x] ~~Player Cooldown Bar (visible when necessary)~~
- [x] ~~Enemy Health Bar~~
- [x] ~~Play/Pause button~~

Tests

- [x] ~~Does the HUD display accurate data for the health of either character?~~
- [x] ~~Does the HUD display a stamina bar for the player, updated in real time?~~
- [x] ~~Does the HUD dynamically display the player's cooldown as a bar?~~
- [x] ~~Does the HUD display a play/pause button is the top left corner for the user?~~
- [x] ~~Do all of the bars show relevant information which is consistently updated?~~

Methods

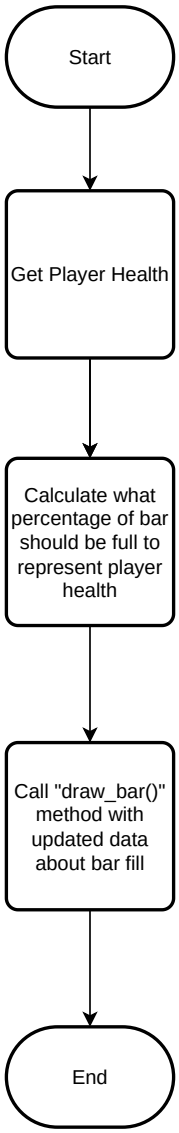
Many of these methods will be reused/repurposed for other classes. For example, I will use `Player Health Bar Draw` for other bars such as player stamina and enemy health. The same goes for the update function.

Bar Draw

```
CLASS Bar
     FUNCTION Bar(fill)
          THIS.fill = fill
     ENDFUNCTION

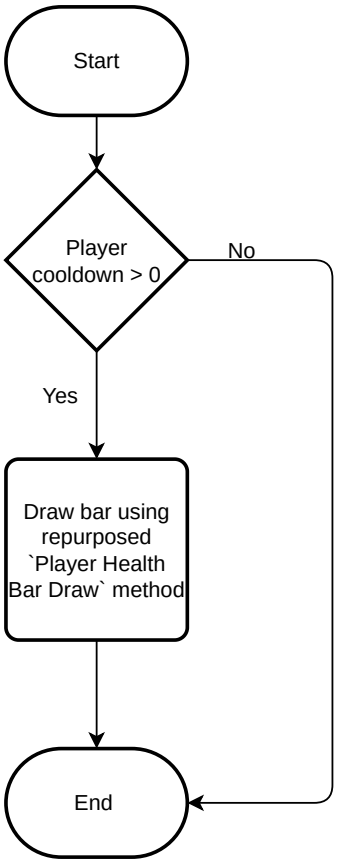

     FUNCTION draw()
          Bar health_bar = new Bar()
          health_bar.fill(fill, "red")
     ENDFUNCTION
ENDCLASS
```

Player Health Bar Update

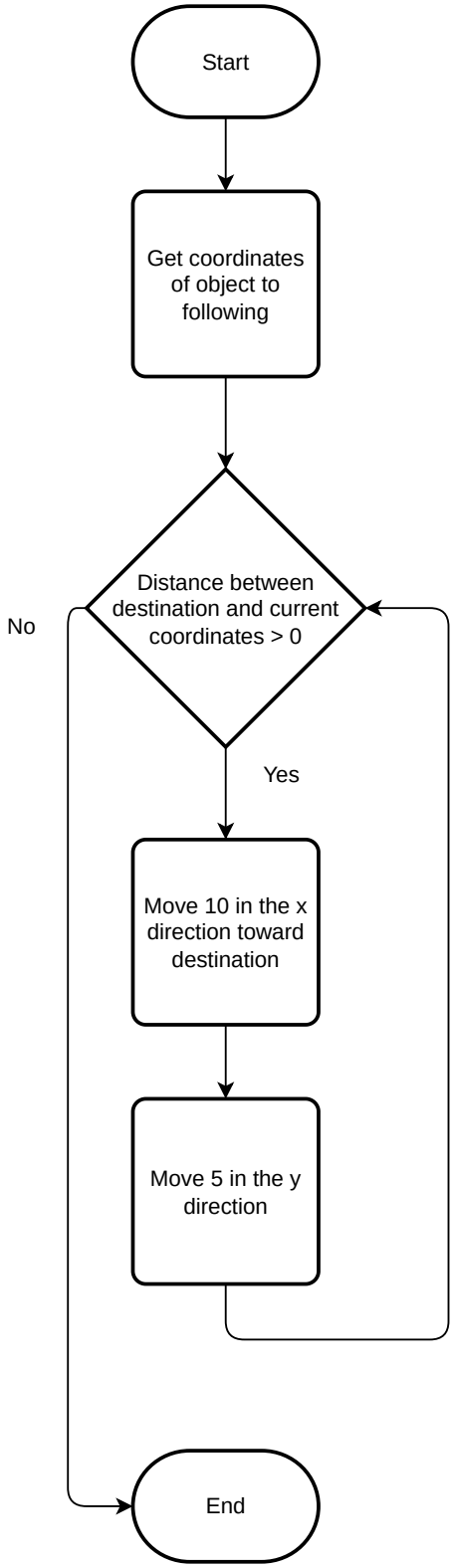


Player Cooldown Bar

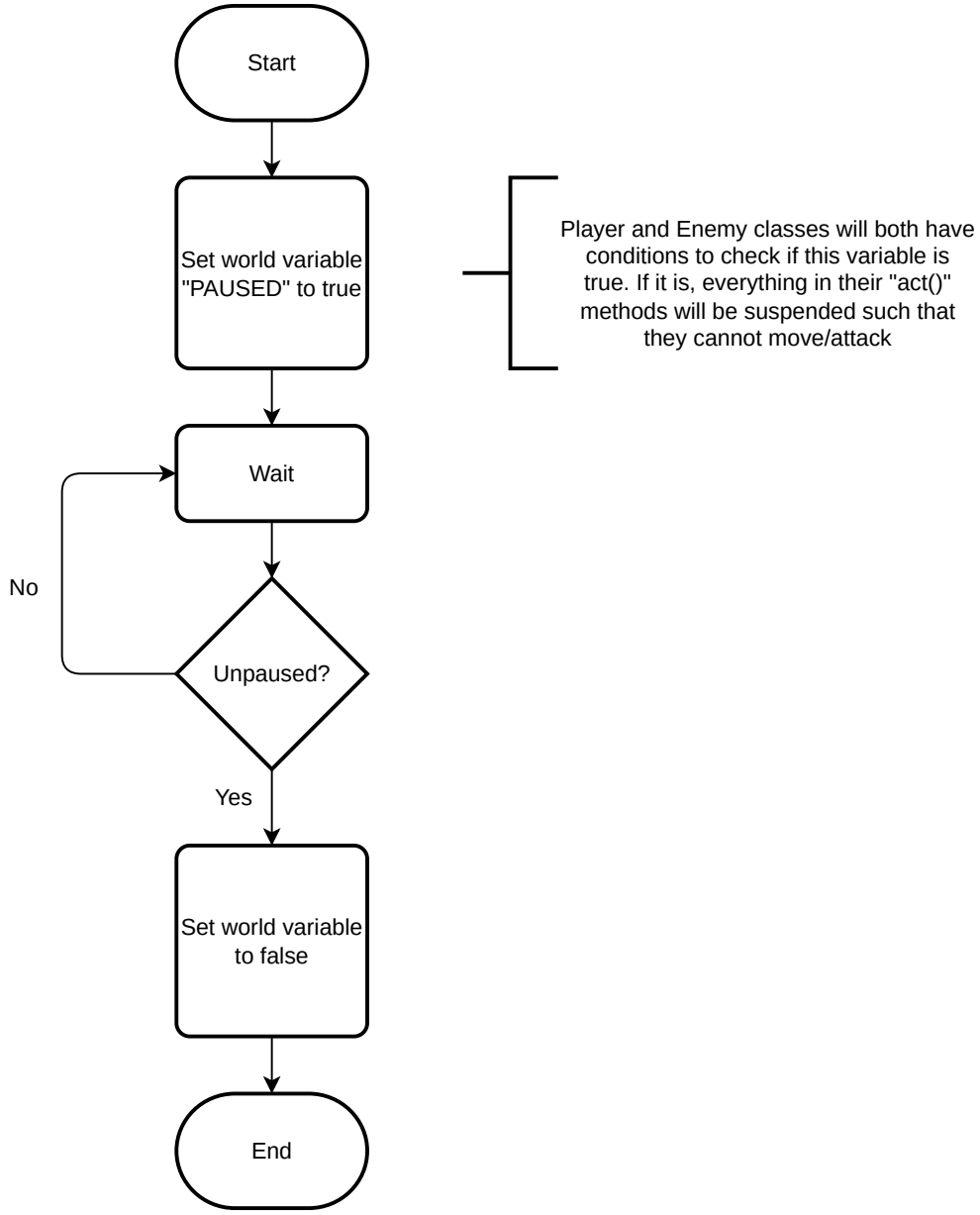
The following is an example of how a bar might be constructed.



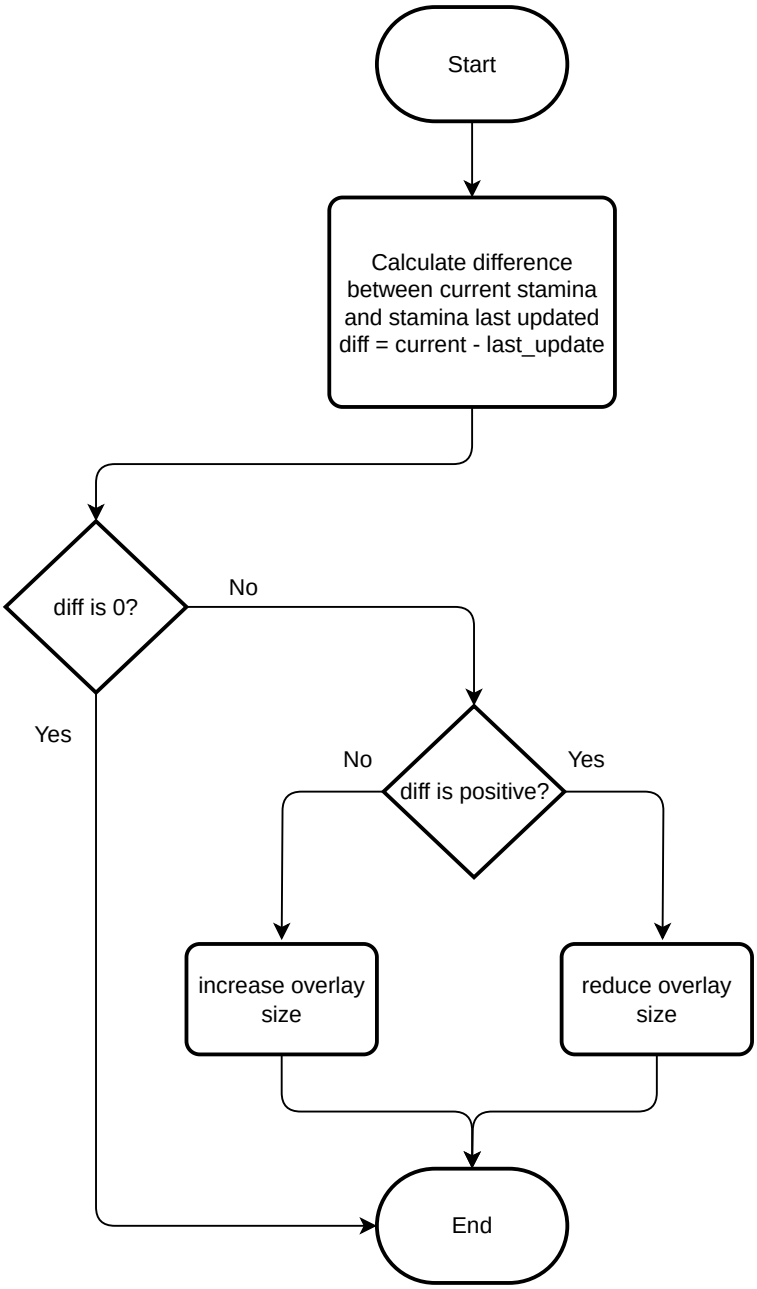
Bar Following Player/Enemy


Start
Get coordinates of object to following
Distance between destination and current coordinates > 0
No
Yes
Move 10 in the x direction toward destination
Move 5 in the y direction
End

Pause

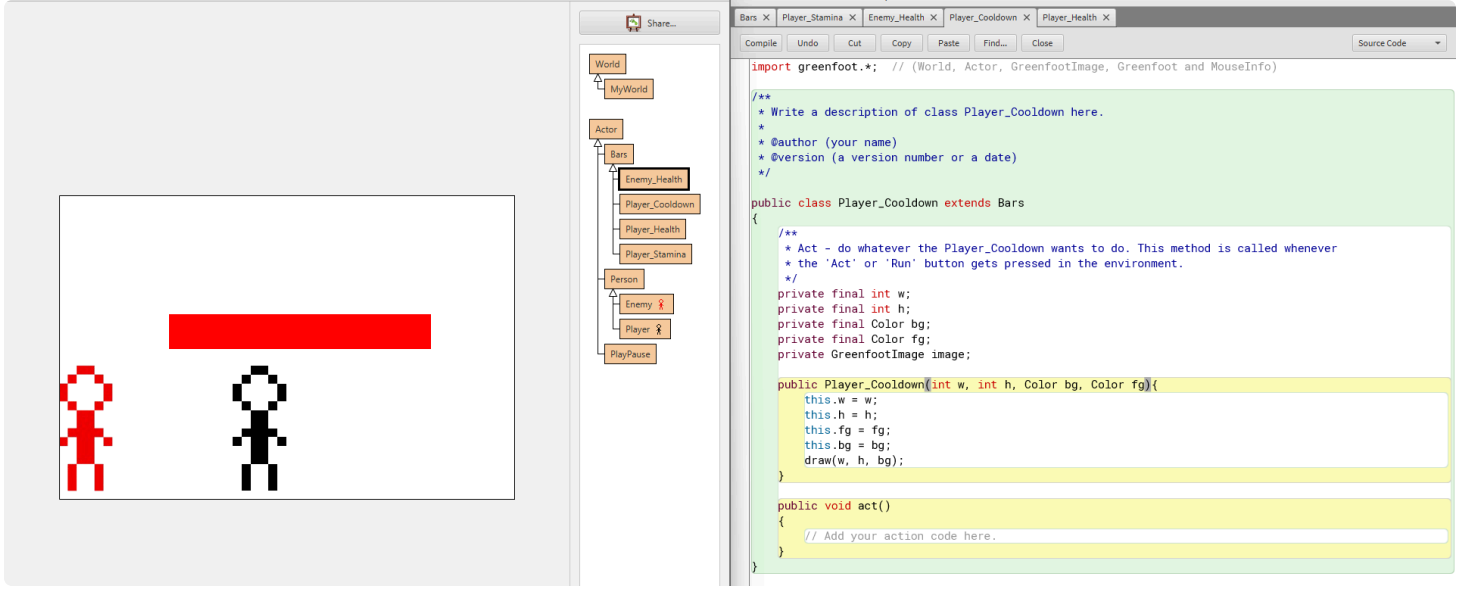


Stamina Bar Update



Notes

Creating the Bars



I decided that it would be best to use a parent class for the bars, meaning that they could all inherit methods such as `draw()` and `update()`. The above image shows me testing the constructor of `enemy_health` bar, producing a large red bar. This point in development was part of my iterative testing, allowing me to ensure that I would be able to create a bar of any dimensions and colour.

Bar Movement Method and Its Problems

```
public void follow(Person p){
        // passes person such that can follow with p.getX() and p.getY()
        int x = getX();
        int y = getY();
        int next_x = p.getX();
        int next_y = p.getY() + this.hover_space;
        // the goal is to have a slow-ish speed at which to follow the person over
their head
        // this will thus have a separate function called `move_request()` to
handle gradual movement
        // via the act method.
        int tdx = x - next_x; // total displacement x
        int tdy = y - next_y;
        // these values will be constantly updated via the act method
        // move_request() will be called to move incrementally to the next_x/y
location
        while(x != next_x && y != next_y){
            move_request(10, 10);
        }
    }

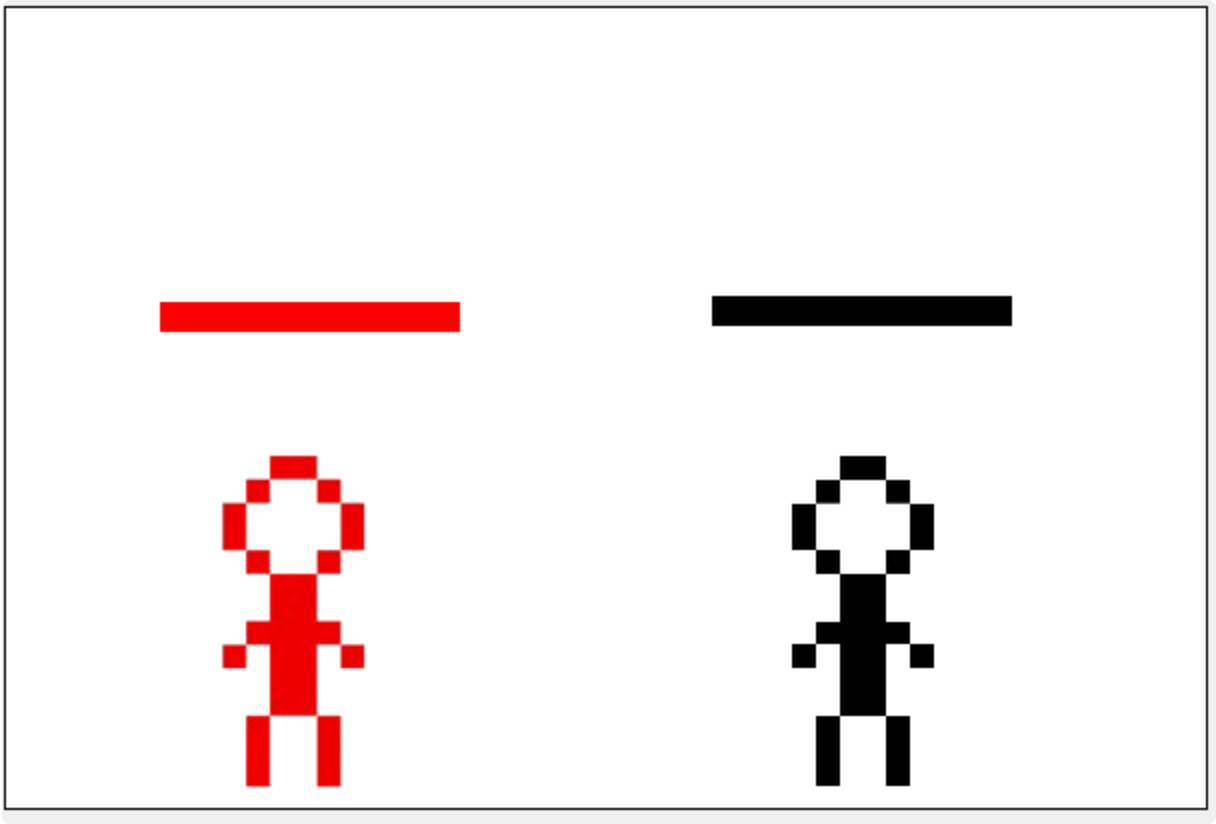
    public void move_request(int dx, int dy){
        setLocation(getX()+dx, getY()+dy);
    }
```

This is the state of the bar `follow()` method as of the 12th of October. As explained in the comments, the method calls the `move_request()` method to move the bar. This avoids errors which I have previously had with motion such as jumping or changing image, where Greenfoot finishes the entire loop instantly, removing the smooth movement which I would like.
I have since fixed this with the following solution:

```
    public void follow(Person p){
        // passes person such that can follow with p.getX() and p.getY()
        int x = getX();
        int y = getY();
        int next_x = p.getX();
        int next_y = p.getY() - hover_space;
        // the goal is to have a slow-ish speed at which to follow the person over
their head
        // this will thus have a separate function called `move_request()` to
handle gradual movement
        // via the act method.
        int tdx = x - next_x; // total displacement x
        int tdy = y - next_y;
        // these values will be constantly updated via the act method
        // move_request() will be called to move incrementally to the next_x/y
location
        if(x != next_x && y != next_y){
            if(x > next_x){
                // next to the left
                move_request(-10, 0);
            }
            if(y > next_y){
                // next higher up
                move_request(0, -10);
            }
            if(y < next_y){
                move_request(0, 10);
            }
            if(x < next_x){
                move_request(10, 0);
            }
        }
    }

    public void move_request(int dx, int dy){
        setLocation(getX()+dx, getY()+dy);
    }
```

This fix allowed the bars to properly follow the player and enemy as below:



Damage Fixing

Whilst testing my bars' code, I realised that due to a poor choice of method for damaging an opponent, damage is only truly applied if both player and enemy are directly intersecting. This is because of the method which I have used (`getOneIntersectingObject(Enemy.class)`) one returns `enemy` if they are intersecting. To remedy this, I will change my `punch()` and `kick()` method from

```
    private void punch(){
        if(stamina >= STAMINA_DRAIN_PUNCH){
            if(cooldown == 0){
                try{
                    Enemy enemy = (Enemy) getOneIntersectingObject(Enemy.class);
                    enemy.damage(DAMAGE_PUNCH);
                    enemy.knockback(KNOCKBACK_PUNCH);

                } catch(Exception e){

                }

                setImage("player_punch.png");
                stamina = stamina - STAMINA_DRAIN_PUNCH;
                //cooldown(COOLDOWN_PUNCH, this);
                cooldown = cooldown + Person.round(COOLDOWN_PUNCH*60, "###");
            }
        }
        imgState = 0;
```

```
    }

    private void kick(){
        if(stamina >= STAMINA_DRAIN_KICK){
            if(cooldown == 0){
                try{
                    Enemy enemy = (Enemy) getOneIntersectingObject(Enemy.class);
                    enemy.damage(DAMAGE_KICK);
                    enemy.knockback(KNOCKBACK_KICK);
                } catch(Exception e) {

                }

                setImage("player_kick.png");
                stamina = stamina - STAMINA_DRAIN_KICK;
                //cooldown(COOLDOWN_KICK, this);
                cooldown = cooldown + Person.round(COOLDOWN_KICK*60, "###");
            }
        }
        imgState = 0;
    }
```

to

```
    private void punch(){
        if(stamina >= STAMINA_DRAIN_PUNCH){
            if(cooldown == 0){

                if(MyWorld.enemy.getX() < 5){
                    MyWorld.enemy.damage(PUNCH_DAMAGE);
                    MyWorld.enemy.knockback(KNOCKBACK_PUNCH);
                }

                setImage("player_punch.png");
                stamina = stamina - STAMINA_DRAIN_PUNCH;
                //cooldown(COOLDOWN_PUNCH, this);
                cooldown = cooldown + Person.round(COOLDOWN_PUNCH*60, "###");
            }
        }
        imgState = 0;
    }

    private void kick(){
        if(stamina >= STAMINA_DRAIN_KICK){
            if(cooldown == 0){

                if(MyWorld.enemy.getX() < 5){
                    MyWorld.enemy.damage(KICK_DAMAGE);
                    MyWorld.enemy.knockback(KNOCKBACK_KICK);
```

```
                }

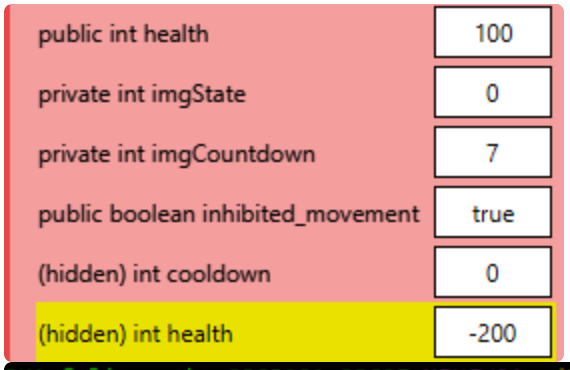
                setImage("player_kick.png");
                stamina = stamina - STAMINA_DRAIN_KICK;
                //cooldown(COOLDOWN_KICK, this);
                cooldown = cooldown + Person.round(COOLDOWN_KICK*60, "###");
            }
        }
        imgState = 0;
    }
```

Variable Shadowing

After a lot of research and smoke testing, I discovered that the root cause of this problem was because of something called variable shadowing (

> ## Variable Shadowing
>
> I learnt about what variable shadowing is and how it happens from this article: https://medium.com/tuanhdotnet/master-variable-shadowing-and-hiding-in-java-uncovering-key-nuances-and-pitfalls-b6eab3d2a41c

). This became obvious when inspecting either the player or enemy characters and finding an extra field which went undeclared in their code:




```
BHA+felix.graham25@R131-22607 MINGW64 ~/CS-NEA/CODE (main)
$ grep -n "health" Player.java
36:    public int health = 100;

BHA+felix.graham25@R131-22607 MINGW64 ~/CS-NEA/CODE (main)
$ grep -n "health" Player.java Person.java
Player.java:36:    public int health = 100;
Person.java:18:    int health;
Person.java:86:        return p.health;
Person.java:97:        p.health = p.health-d;
```

As I found out, this was the result of declaring `health` and `cooldown` in the parent class, despite this being used to write the methods for both player and enemy.

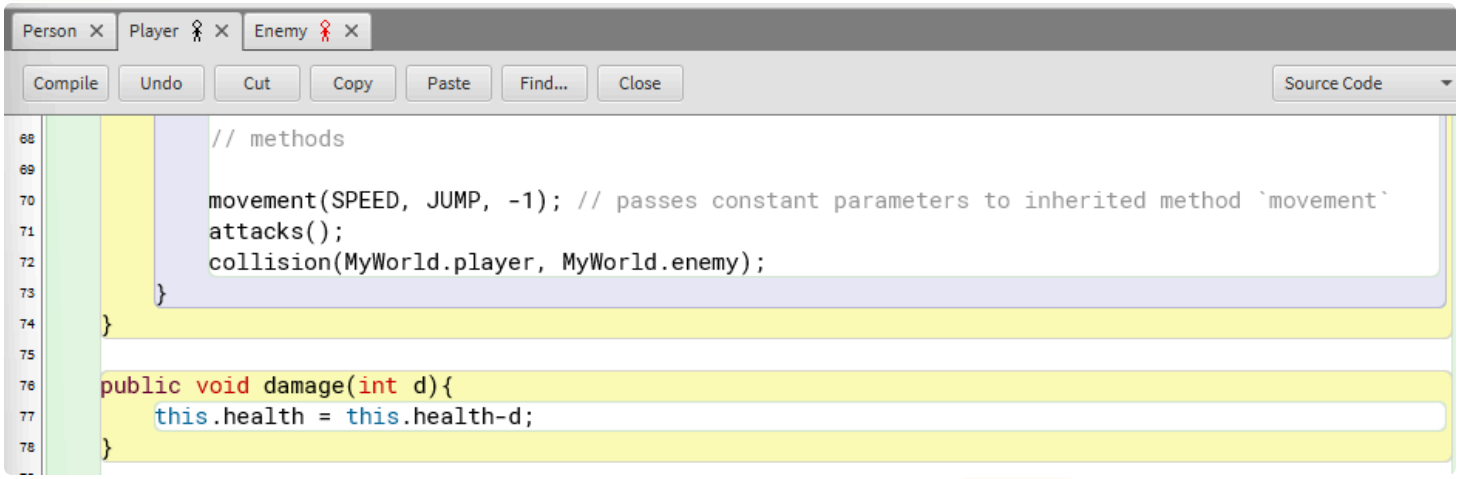
```
public class Person extends Actor
{
    int cooldown;
    int health;

    .
```

```
.
.

public void damage(int d, Person p){
    p.health = p.health-d;
}
```

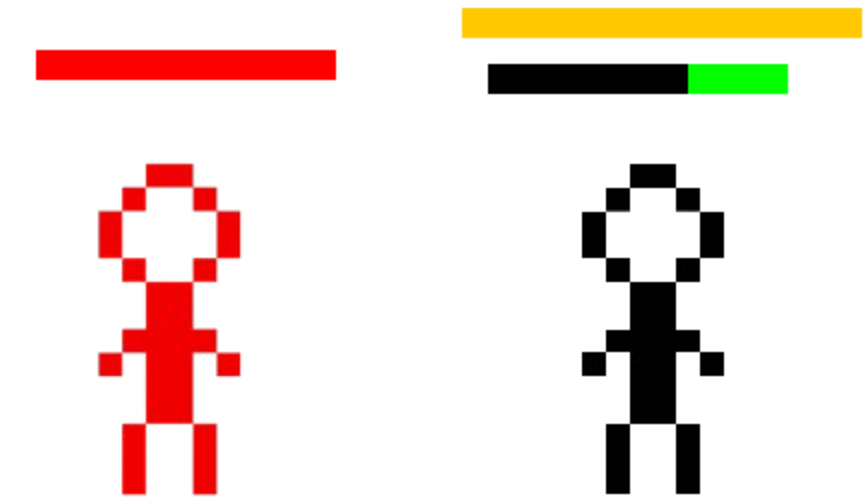
As of 19/10/2026, I have fixed this. I did so by declaring the `damage()` method in both `Enemy` and `Player` classes following a polymorphic approach.



This allowed damage to be applied directly to either Persons' static `health` variable as such:



Because of this, I was able to fully implement a dynamically updating health bar for both classes:



My research about this error comes from the following two sites. No code from either was copied.
https://www.geeksforgeeks.org/java/shadowing-in-java/
https://codingtechroom.com/question/understanding-variable-shadowing-java

Damage Below A Threshold

Whilst testing, I discovered that I had a small logic error for when the value of health is less than 0. This stemmed from the code:

```
private int player_heartbeat(){
     int pc = 100-MyWorld.player.health;
     return pc;
}
```

If `MyWorld.player.health` is less than 0, the following would happen:

$$\text{et } . . = 50$$

$$= 100 \ (50) = 150$$

$$\text{ealth Bar would overfill to } 150$$

To prevent this, I implemented the following check into the `damage` method:

```
public void damage(int d){
     if(this.health-d <= 0){
          this.health = 0;
     } else{
          this.health = this.health-d;
     }
}
```

Bar Fixes

The line

```
int w = round((b.w*(pc/100)), "###"); // rounds
```

in the method `update()` in `Bars` was causing the bars' updating system to be off. Because I had not noticed the logic error when writing this method, a bar would only appear to update when the Person's health had reached 0.

```
public void update(int pc, Color c, Bars b){ // where pc is the percentage of the
bar to be filled
        // calculate width to fill from given percentage
        int w = round((b.w*(pc/100)), "###"); // rounds
        this.image.setColor(c);
        this.image.fillRect(0, 0, w, b.h+5); // addition to height ensures all bar
covered
        this.image.fillRect(0, 0, w*2, b.h+5); // addition to height ensures all
bar covered
    }
```

$$\text{Assume } = 0$$

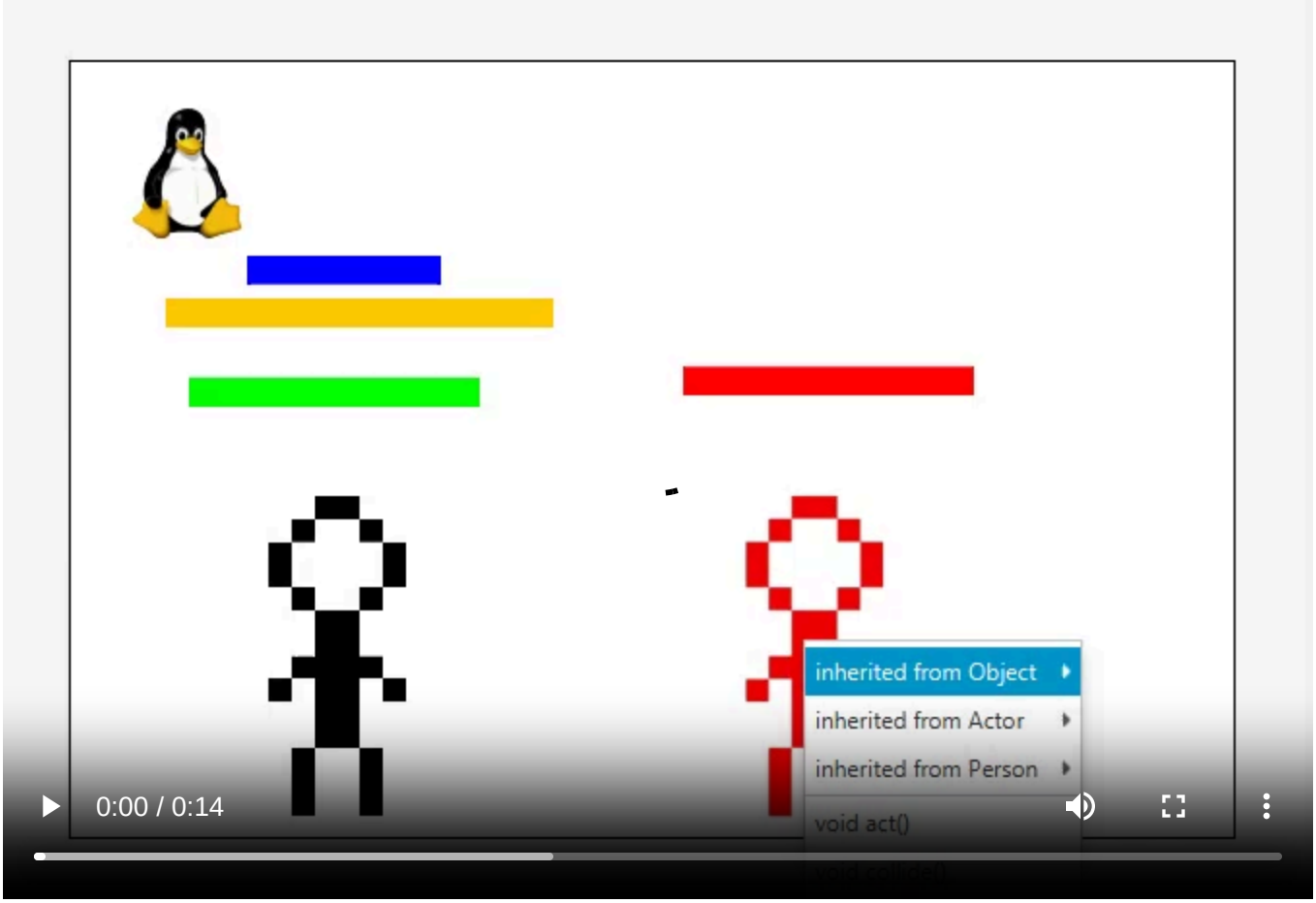
$$\text{Assume } w = 100$$

$$w = n\left(100\ \left(\frac{0}{100}\right), \text{ to sf}\right)$$

ue to the nature of the rounding method,

this does not produce an intended value for w, often leaving it to be 100 or 0.

This made the game hard to play because it was not possible to see either characters' health until they were dead.



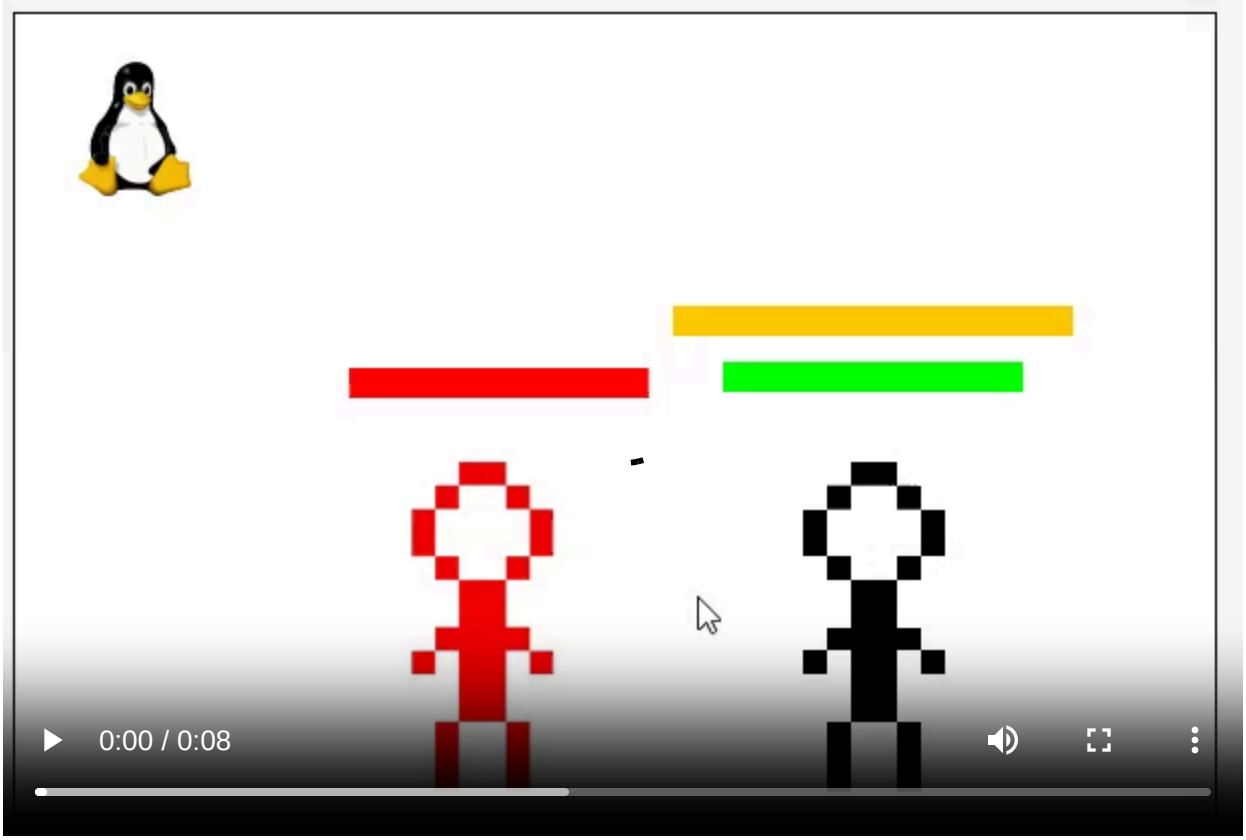
absolutely_epic_combo.mp4

(The player in the video has been given extended range of attack, such that the bug can be properly seen)

I fixed it by removing the unnecessary rounding to yield:

```
public void update(int pc, Color c, Bars b){ // where pc is the percentage of the
bar to be filled
        this.image.setColor(c);
        this.image.fillRect(0, 0, pc*2, b.h+5); // addition to height ensures all
bar covered
    }
```

This allowed accurate updating of health bars

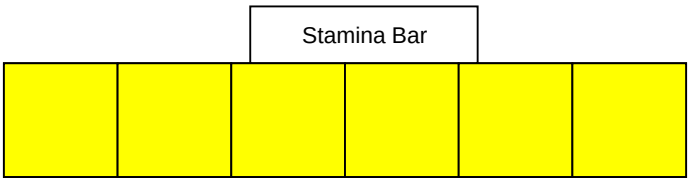


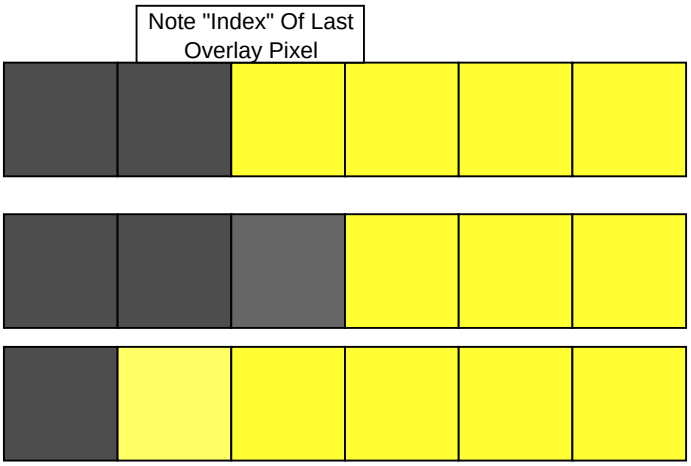
enemy_being_thwarted.mp4

Stamina Bar Update

For the stamina bar to be updated, it should cover more or less of the yellow background with black. This is a unique challenge which was not present with either health bar. With these the black overlay is actually part of the same image, which was a useful design choice because they will not regenerate and the simplicity of this improves the efficiency of the program because only one GreenfootImage item is being updated.
The reason that this would be problematic for `StaminaBar` is that it can not be removed. It is easy to implement `increase_overlay_size()` but difficult to add `decrease_overlay_size()`. Below are some diagrams explaining potential solutions to this problem.

Index System





The idea behind this is that the stamina bar would be split into groups of n pixels and stored inside an array. A pointer would hold the location of the "head" of the overlay, meaning that `increase_overlay_size()` would look like

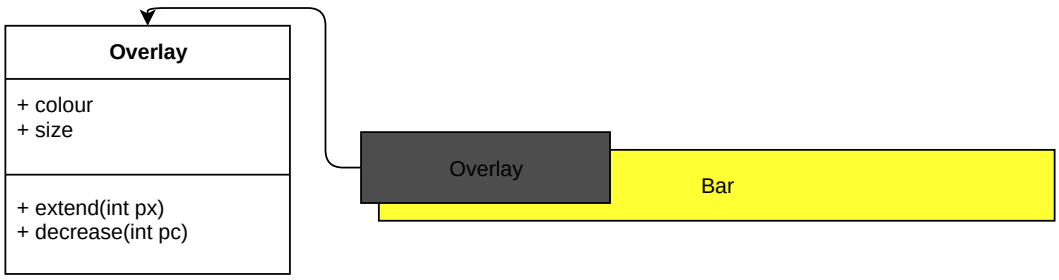
```
PROCEDURE increase_overlay_size(head)
     set_overlay(stamina_bar[head+1]);
END PROCEDURE
```

and `decrease_overlay_size()` like so:

```
PROCEDURE decrease_overlay_size(head)
     set_not_overlay(stamina_bar[head])
     head = head-1
END PROCEDURE
```

The downside of this is that the bar would lack end up sacrificing either efficiency or accuracy for the other.

Separate Overlay



The benefit of this system is it would be very easy to implement, even with the same methods for

following allowing it to be an effective overlay. The downside is that it would be a whole new class which would clutter the environment, make the game less resource efficient, and introduce potential new errors.

In conclusion, I will use option 2. It will be far easier than implementing a stack system and the clutter will be minimal.

Implementation Challenges



I was able to implement `Stamina_Overlay` as a separate class with the same movement method as `Player_Stamina`. This allowed it to be in the same place at the same time. Unfortunately, I had a null pointer error when running the program. This referenced the following code:

```
public class Stamina_Overlay extends Bars {
     // irrelevant code ommitted

     public int w;
     public final int h;
     private final Color c;
     public static GreenfootImage image;

     public void increase(int amount){
          this.w = this.w + amount;
          if(w > 0){
          } else {
               w = 0;
          }
          image.fillRect(0, 0, this.w, this.h);
     }
}
```

With the offending line:

```
image.fillRect(0, 0, this.w, this.h);
```

To fix this I tried making all the data static and reference the object, like `Player_Stamina` would reference the `increase()` method by object.

```
public class Stamina_Overlay extends Bars {
     // irrelevant code ommitted

     public static int w;
     public static int h;
     private static final Color c;
     public static GreenfootImage image;

     public void increase(int amount){
          Stamina_Overlay.w = Stamina_Overlay.w + amount;
          if(w > 0){
          } else {
               w = 0;
          }
          Stamina_Overlay.image.fillRect(0, 0, Stamina_Overlay.w,
Stamina_Overlay.h);
     }
}
```

But this didn't seem to help. I then did the opposite, hoping that referencing the class with `this.{}` with the same result.
In the end I fixed it by properly instantiating `this.image` with the GreenfootImage constructor.

```
public Stamina_Overlay() {
     this.w = 1;
     this.h = Player_Stamina.h;
     this.c = Player_Stamina.fg;

     this.image = new GreenfootImage(this.w, this.h);

     MyWorld.s = this;
}
```

The next problem was that the bar would not update both ways. The above code allowed it to increment accurately, but when I implemented something similar for decrementation, neither would work. I worked out that this was due to the image defined with

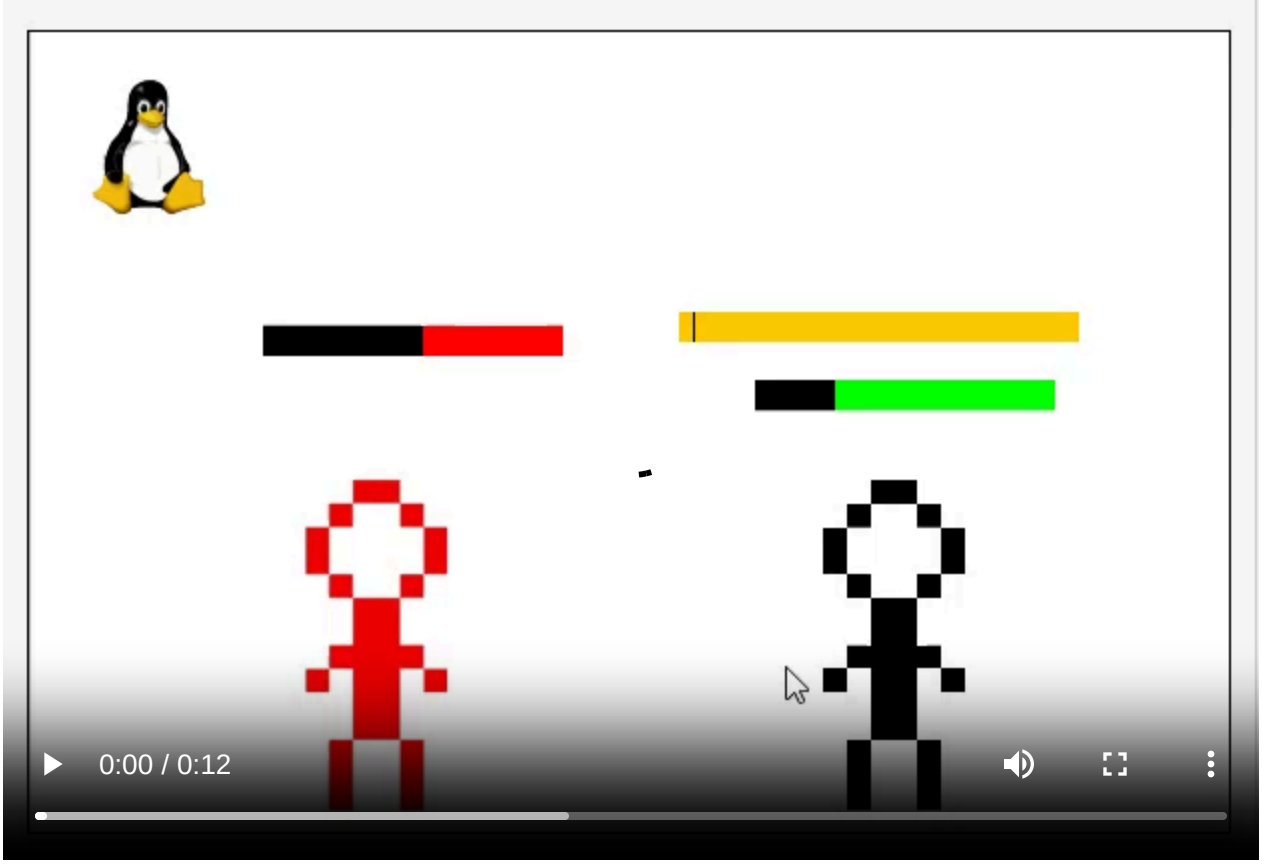
```
public Stamina_Overlay(){
     // irrelevant code ommitted
     this.image = new GreenfootImage(this.w, this.h);
}
```

```
// and then being referenced by both methods with
public void decrement(int amount){
     this.w -= amount;
     this.image.fillRect(0, 0, this.w, this.h);
}
public void increment(int amount){
     this.w += amount;
     this.image.fillRect(0, 0, this.w, this.h);
}
```

This was problematic because the variable was only being modified with local scope. This would not therefore function as intended.
I fixed this, with optimisations, by removing `increment()` and `decrement()` and replacing them with `augment()`. This would take a signed integer as a parameter to alter.

```
public void augment(int amount){
     this.w = this.w + amount;
     if(w > 0){
     } else{ // this is innefficient but improves readability
          w = 0;
     }
     setImage(new GreenfootImage(this.w, this.h));
     this.image = getImage();
     this.image.setColor(this.c);
     this.image.fillRect(0, 0, w*2, this.h+5); // with w*2 because stamina is out
of 100 and stamina bar has width 200. h+5 for better coverage
}
```

By redefining the image, we gain full mutability, which was hard to access through the global variable with two methods. Here is the result:

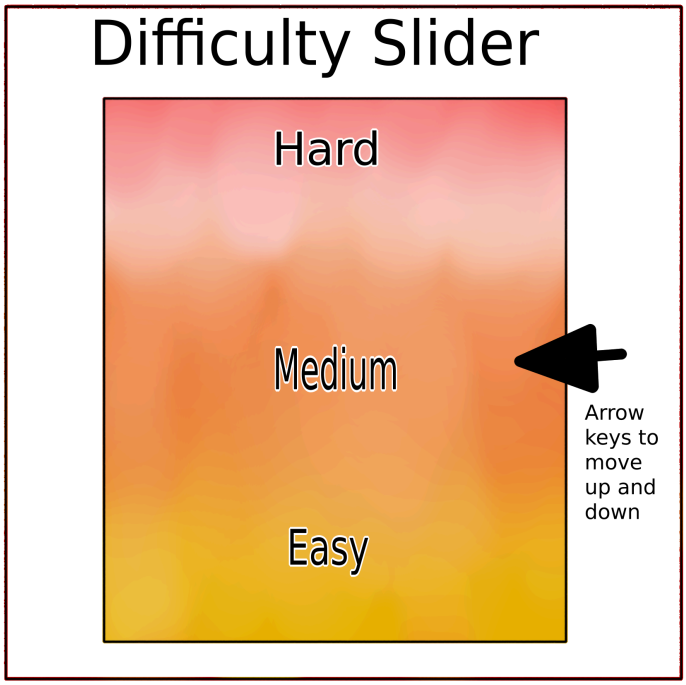


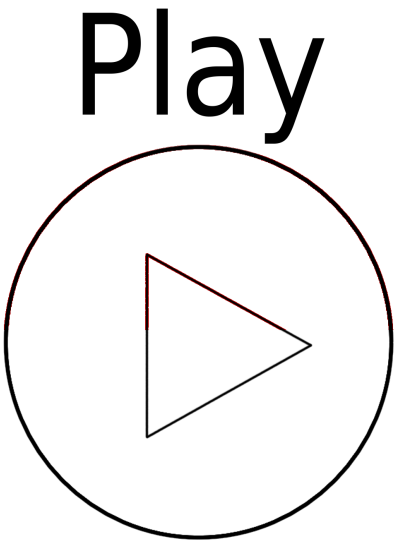
[stamina_overlay_working.mp4]
This concluded stage 4.

Stage 5

This stage is about designing and creating the start menu. It should feature a difficulty slider and start button. I plan to accomplish this my having a separate `World` class called `Start`.

Bar Fight





To Do List

Features

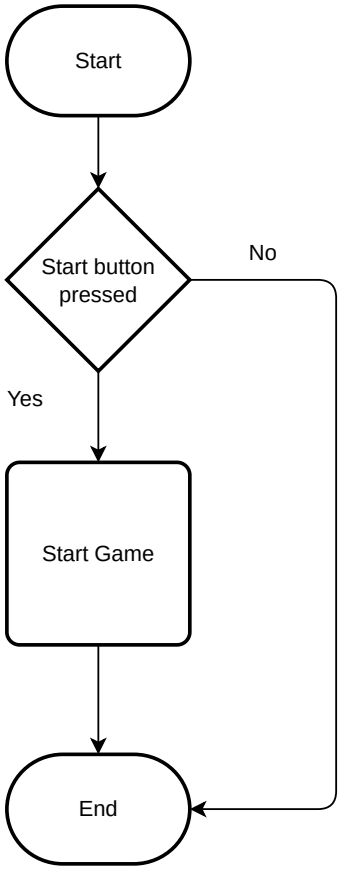
- [x] ~~Start Screen Menu~~
- [x] ~~Difficulty Slider~~
- [x] ~~Difficulty Value~~
- [x] ~~Start Button~~

Tests

- [x] ~~Start Button starts game~~
- [x] ~~Start Screen Menu loads first when running the game~~
- [x] ~~Start Screen Menu holds Start Button and Difficulty Slider~~
- [x] ~~Difficulty Slider alters Difficulty Value~~

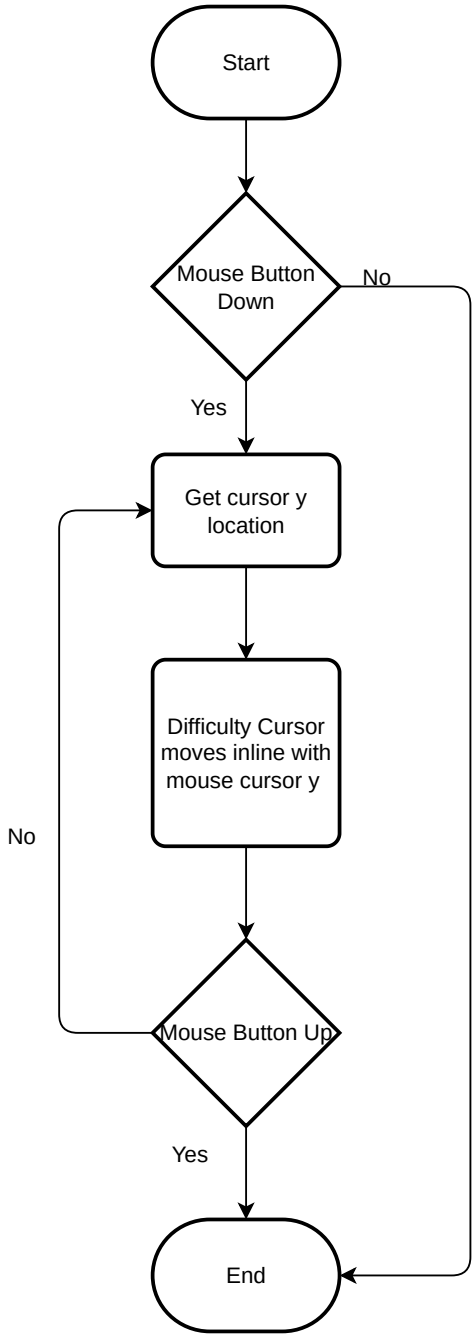
Methods

Start Game

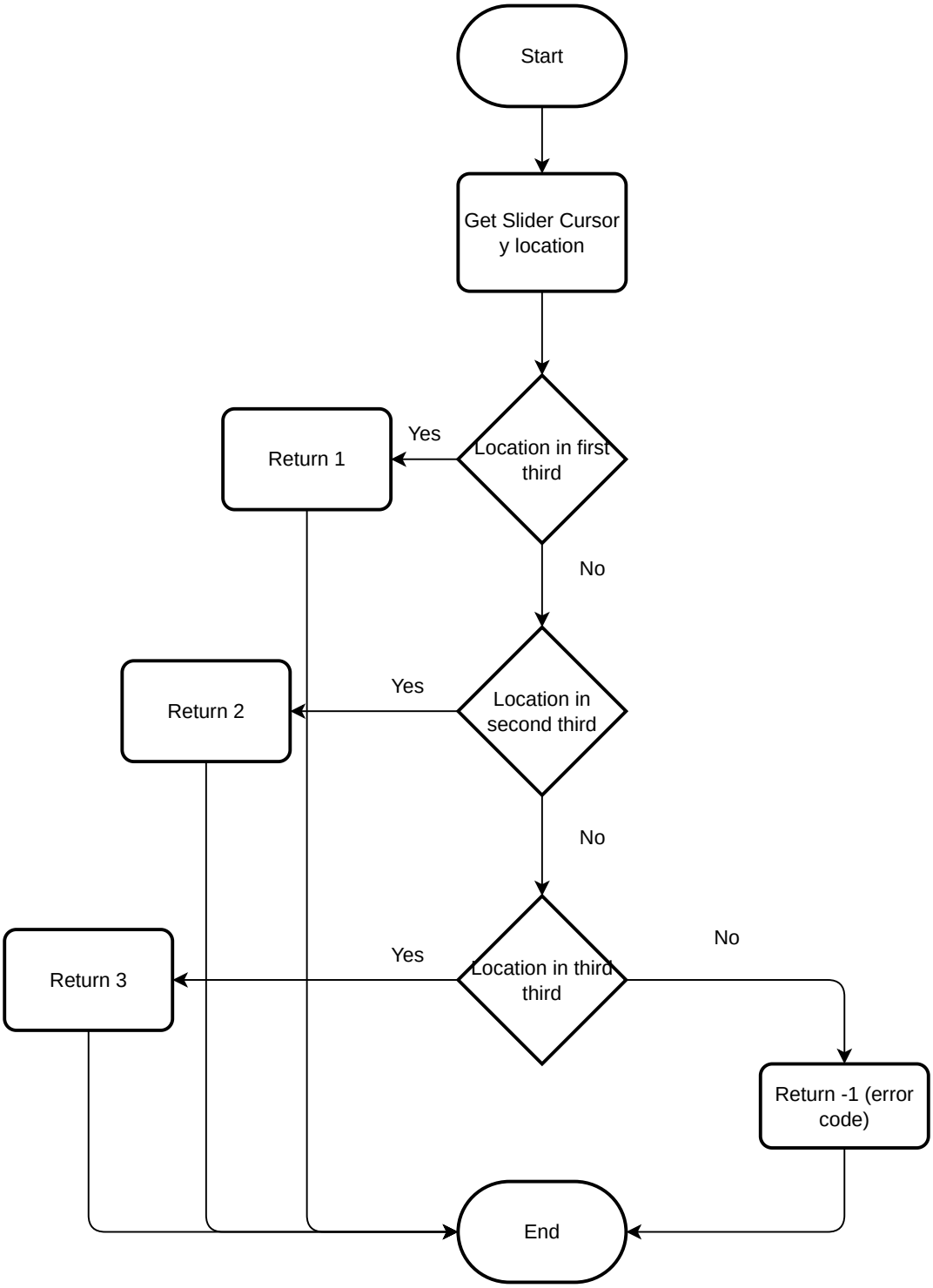


Difficulty Slider

Cursor Follow



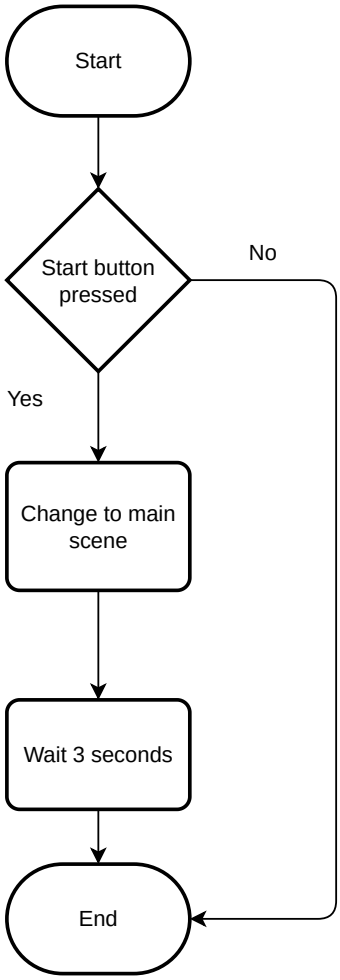
Update Value



Notes

Start Button Alteration

After implementing my start button, I realised that the transition from menu to action is too fast. To remedy this, I came up with the following amendment to the start game flowchart:



The `wait 3 seconds` would make use of the Greenfoot `act()` method being called 60 times per second. The waiting would happen because after it had changed worlds, it would set the global `paused` to `true` whilst 3 seconds has not passed.
This idea was sadly flawed due to what turned out to be Greenfoot's entity permanence. After the start button changed the world, the object was destroyed and thus - having paused the world - would not resume. To fix this, I introduced two new methods into the `MyWorld`, being `wait_please()` and `are_we_unpausing()`. The static method `wait_please()` takes `t` as a parameter, which is the number of `act()` calls. This is used as a setter to modify a variable in `MyWorld` called `wait`.
Following this, the method `are_we_unpausing()` is called each `MyWorld act()` run. It is a very simple check to see if the `wait` variable is equal to or less than 0. I have decided that it should be its own method for simplicity in the act method and ease of reading, with both new methods next to each other. This produced working code as follows:



[start_delay.mp4]

Difficulty Slider

This went without much trouble. I even had time to implement a colour change for more visual feedback to the user based on their chosen difficulty level.



[difficulty_slider.mp4]
It can be moved both with the mouse and keyboard.
With this finished I was able to move onto my testing, where all of my tests passed.

Stage 6
In this stage I had previously aimed to add the the attacks feature in my game. However I have previously made the decision to incorporate this into an earlier stage along with the addition of the character. I chose to do this because it made testing additional features such as the health-bars immensely easier.
As a result of this, I plan to re-purpose this stage into several fixes between the player and enemy, focusing on elements such as collision and tracking. My resultant features and test plan are below. The majority of these fixes feature in my final test plan and have not been assigned a proper stage; this stage should fix that.

To Do List

Features

- [ ] Player and Enemy collision
- [ ] Enemy-Player tracking
- [ ] Accurate attack ranges

Tests

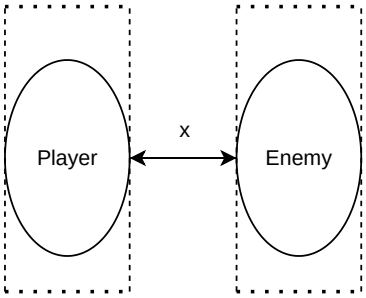
- [ ] One character moving into the other (stationary) pushes them
- [ ] Characters moving toward each other are stopped at an appropriate distance
- [ ] Collision is applied in both directions for each character
- [ ] A jumping character cannot jump into another
- [ ] Enemy can detect player when they are to their left
- [ ] Enemy can detect player when they are to their right
- [ ] An attack is only applied when the target is within an appropriate range

Methods

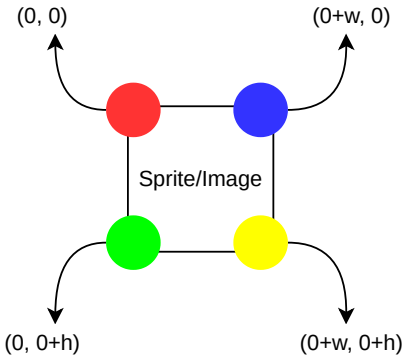
Collision

Unlike some other methods, the solution for this is not obvious. To develop a method to implement effective collision, I had to explore several possible methods of execution, as demonstrated by several diagrams below.

Collision by X-Value



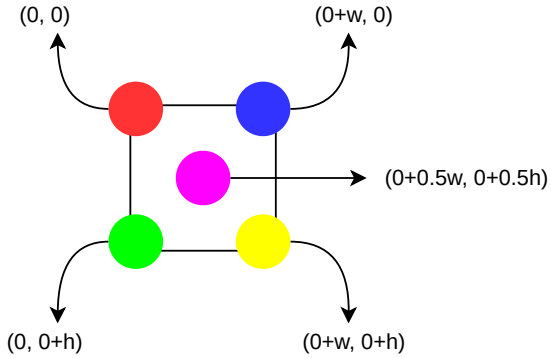
The issue is that the given x-value would only be for one side. My current knowledge of Greenfoot leads me to believe that a sprite/image's coordinate system works like this:



This would make this approach problematic for judging accurate displacement.

∆Centre

We can solve the above problem with an approach like this:

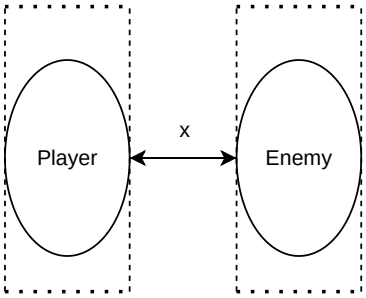


This solution would allow for each x 'bar' to be calculated with

$$X_n = \frac{1}{2}w \pm \frac{1}{2}w + x$$

where X_n is the bar; w is the sprite width; x is the value returned with ‘getX()’

We could then calculate collision with the same method as proposed above:



Where x would be an adjustable variable for collision difference.
With this, each actor involved in collision would have its own public variables for `x1` and `x2`.

```
CLASS Player
     ...
     FUNCTION update_bars()
          THIS.x1 = 0.5(getWidth())-0.5(getWidth)+getX()
          THIS.x2 = 0.5(getWidth())+0.5(getWidth)+getX()
     ENDFUNCTION
ENDCLASS

CLASS Person
     ...
     FUNCTION get_collision(Person player, Person enemy)
          IF player.getX() > enemy.getX() THEN
               dx = player.x1 - enemy.x2
          ELSE
               dx = enemy.x1 - player.x2
          ENDIF

          RETURN dx
     ENDFUNCTION
ENDCLASS
```

The `collide()` method would work like so

```
CLASS Person
     ...
     FUNCTION collide(Person p, Person e)
          dx = get_collision
          IF p.getX() > e.getX() THEN
               WHILE dx <= collision_limit THEN
                    p.setLocation(p.getX()+1)
                    e.setLocation(e.getX()-1)
               ENDWHILE
          ELSE
               WHILE dx <= collision_limit THEN
                    p.setLocation(p.getX()-1)
                    e.setLocation(e.getX()+1)
```

```
                ENDWHILE
        ENDFUNCTION
ENDCLASS
```

The function is based on a heuristic, assuming that a full collision where `Player` is mapped entirely onto `Enemy` never occurs. This also fulfills the feature of pushing:

$$\text{Assume a particle A is moving with velocity } +5\text{m/s, with a second particle B stationary.}$$

$$\text{The resultant collision force acting on both particles is -1m/s}$$

$$\text{Therefore if we assume the driving force for particle A is constant and thus has acceleration of 0,}$$

$$\text{we can see that the new velocity of B upon collision would be -1m/s, thus moving it back, whilst A's}$$

$$\text{would be 4m/s, keeping it moving forward at a reduced weight.}$$

Notes

References

references

Image Resizing

I have used https://imageresizer.com/ to resize my player sprites at 14:28 on the 29/09/2026. I did this because after drawing them on my laptop, they were too large to have effectively in my game. Had I not done this, the player sprite would have been too large to play and therefore been a detriment to player experience.

How to round to an integer

https://www.geeksforgeeks.org/java/java-program-to-round-a-number-to-n-decimal-places/
Accessed at 10:28 on the 28th of September 2026.

Notes

I looked at this article because I needed to convert the product of `t*60`

```
public void cooldown(double t){
    this.cooldown = t*60;
```

where `t` is a double. This is because I would like to have the cooldown of my punch attack to be 0.2 seconds. The issue with this is that Greenfoot cannot iterate for a fractional number, something becoming problematic in the tail end of that method:

```
for(int c = this.cooldown; c>=0; c--){
    this.cooldown--;
```

```
}
```

To resolve this, I decided to round the product of `t*60` to the nearest whole number, which is why I looked up this article.

From this article, I learnt from the example:

```
import java.math.RoundingMode;
import java.text.DecimalFormat;

public class GFG {
    public static void main(String[] args) {
        double number = 9.97869896;

        DecimalFormat df = new DecimalFormat("#.####");
        df.setRoundingMode(RoundingMode.FLOOR);

        System.out.println(df.format(number));
    }
}
```

how to use the `DecimalFormat` library to round a decimal. This led me to come up with the following for my own situation:

```
import java.math.RoundingMode;
import java.text.DecimalFormat;

public void cooldown(double t){
     double unrounded_cooldown = t*60;
     DecimalFormat df = new DecimalFormat("###"); // where unused digits are 0
     df.setRoundingMode(RoundingMode.CEILING); // rounds up from t*60
     this.cooldown = df.format(unrounded_cooldown); // sets the cooldown to
appropriate integer
     for(int c = this.cooldown; c>=0; c--){ // same logic as if it were an int
       this.cooldown--;
    }
}
```

Accessing Data From Another Class

Whilst implementing the enemy `Damage Multiplier`, I forgot how to reference data from another class, being `Player`. I looked at https://www.greenfoot.org/topics/2241 to resolve this. This allowed me to implement the following

```
Player p = (Player) getWorld().getObjects(Player.class).get(0);
```

```
int x = p.getX(); // test
```

Note, I ended up not using this after 05/10/2026 when I decided to pass the player class directly to the enemy class, thus removing this error.

Removing Image Backgrounds

I used https://remove.bg to remove the backgrounds of the player sprites. This means that their previously white backgrounds do not block out the other player sprite. Had I not done this it would have been hard to see the other character when they were close.

Using `try catch` to avoid Null Pointer Errors

As a result of my player code's attack methods defining the enemy with

```
Enemy enemy = (Enemy) getOneIntersectingObject(Enemy.class);
```

my game would often throw a null pointer exception because the enemy object's methods were to be called. This caused the problem because if the player was not intersecting with the enemy, no object would be returned from `getOneIntersectingObject()` and the rest of the code

```
enemy.damage(DAMAGE_KICK);
```

would not work.
I knew that I could fix this with either `try/catch` or an `if` clause to check if `Enemy` was null. I chose to use the first of these due to it allowing a further split. An if statement would have still executed the code after it whilst this method would have two possible outcomes unique to both scenarios.
I have used try/catch before in other languages such as C and Python, but I needed to look up how to implement something similar in Java. I learnt how from this web page:

> **Link**
>
> https://www.w3schools.com/java/java_try_catch.asp

Variable Shadowing

I learnt about what variable shadowing is and how it happens from this article:
https://medium.com/tuanhdotnet/master-variable-shadowing-and-hiding-in-java-uncovering-key-nuances-and-pitfalls-b6eab3d2a41c